

SYNOPSIS

Title of the Project Proposal:

ZEN-ZEE STYLE

Name: Md Arman Ansari

Enrollment: 2200279405

Programme: BCA

Course Code: BCSP-064

Study Centre Code:0758P

Regional Centre Code: 07: DELHI 1

E-mail ID: arman.ansari.2004786@gmail.com

Mobile Number: 7838350723

CONTENT

1. TITLE & INTRODUCTION OF PROJECT	3
1.1. TITLE	3
1.2. INTRODUCTION	4
2. OBJECTIVE OF PROJECT	5
3. PROJECT CATEGORY	7
4. DRAWBACK OF THE EXISTING SYSTEM	8
5. SYSTEM REQUIREMENT SPECIFICATION	10
6. ANALYSIS OF PROJECT	11
6.1. DATA FLOW DIAGRAM	11
6.2. ENTITY RELATIONSHIP DIAGRAM	14
6.3. FLOWCHART	15
6.4. TABLE ARCHITECHTURE	16
6.5. CLASS DIAGRAM	18
7. MODULE DESCRIPTION	19
8. GANTT CHART	22
9. TESTING OF THE PROJECT	23
10. SECURITY MECHANISM	24
11. FUTURE SCOPE & FUTURE ENHANCEMENT	26
12. LIMITATIONS OF THE PROJECT	28
13. BIBLIOGRAPHY	30

TITLE

"ZEN-ZEE STYLE E-Commerce Platform: An Intelligent Shopping Experience with Smart Product Discovery and Contextual Chatbot Assistant"

The ZEN-ZEE STYLE E-Commerce Platform is a modern online shopping system that uses artificial intelligence to improve the customer experience. It features an Intelligent Discovery Engine for personalized product recommendations and a Contextual Chatbot for real-time customer support. The platform makes online shopping more interactive, personalized, and user-friendly.

INTRODUCTION

The rapid growth of information technology and internet services has transformed the way people purchase goods and services. E-commerce platforms have become an essential part of modern business by enabling users to buy products online with convenience and efficiency. However, many existing e-commerce systems still lack interactive support, personalized assistance, and intelligent user engagement, which often results in poor user experience and customer dissatisfaction.

ZEN-ZEE is a web-based e-commerce application developed to address these limitations by providing a modern and interactive online shopping platform. The system integrates essential ecommerce functionalities such as user registration, product browsing, shopping cart management, secure payment processing, and order tracking. In addition to these standard features, ZEN-ZEE includes **AI-assisted components**, mainly in the form of an integrated chatbot and rule-based recommendation support, to enhance user interaction.

The AI component in ZEN-ZEE is not designed as a complex artificial intelligence model but rather as an **AI-integrated assistance system**. The chatbot helps users by answering product related queries, guiding them during navigation, and providing real-time assistance during the shopping process. This approach improves usability while keeping the system realistic and feasible within the academic scope of a BCA project.

The system is designed using a structured and modular architecture, which is clearly represented through system diagrams such as Data Flow Diagrams (DFD), Entity Relationship Diagrams (ERD), table architecture, and module diagrams. These diagrams explain how data flows within the system and how different components interact with each other.

The primary purpose of the ZEN-ZEE project is to design and implement a reliable, secure, and user-friendly e-commerce platform with AI-assisted interaction, demonstrating practical knowledge of web development, database management, and system design.

OBJECTIVE OF PROJECT

The main objective is to create an intelligent e-commerce platform that uses AI to enhance product discovery and customer support. The specific goals are:

1. To Provide Intelligent Product Discovery

Implement an AI-powered recommendation system that analyzes user preferences, browsing patterns, and trending products to automatically suggest relevant items. This saves time and helps customers find what they need quickly.

2. To Enable Real-Time Customer Support Through AI Chatbot

Integrate a contextual chatbot that understands customer queries in natural language and provides instant help with products, sizing, materials, and purchasing guidance 24/7.

3. To Facilitate Personalized Shopping Experiences

Analyze user behavior, preferences, style choices, and budget to deliver highly personalized product recommendations tailored to each customer.

4. To Offer Comprehensive Product Management

Organize products into clear categories (clothing, footwear, accessories, electronics) with advanced filtering by price, brand, color, size, and ratings.

5. To Implement Smart Cart and Wishlist Features

Allow users to add products to cart, save items for later in a Wishlist, and receive notifications about price drops or restocking.

6. To Integrate Secure Payment and Order Management

Support multiple payment methods (cards, UPI, wallets) with secure processing, automatic order tracking, invoice generation, and delivery updates.

7. To Provide Admin Dashboard for Business Management

Give administrators tools to manage products, view sales analytics, track customer behavior, monitor inventory, and make data-driven decisions.

8. To Ensure Data Security and Privacy

Protect user information using encryption, secure authentication, and role-based access control. Comply with data protection standards.

9. To Build a Scalable System for Future Growth

Design the architecture to support future features like mobile apps, advanced analytics, and integration with other services.

Conclusion:

These objectives aim to make the platform comprehensive, user-friendly, and future-ready. By combining AI with good design, the platform will improve the online shopping experience.

PROJECT CATEGORY

Application-Based Web Project (E-Commerce Domain with AI Integration)

This project is a **full-stack web application** in the **e-commerce and artificial intelligence domain**. It combines traditional online shopping features with modern AI technologies.

The technology stack includes:

- **Frontend:** HTML, CSS, JavaScript, React.js
- **Backend:** Node.js with Express.js
- **Database:** MongoDB or MySQL
- **AI Components:** Python for recommendation algorithms, NLP for chatbot

This project demonstrates:

- Complete software development lifecycle
- Web development skills (frontend + backend)
- Database design and management
- AI/ML integration in practical applications
- Security implementation
- User experience design

The platform represents modern e-commerce trends and prepares for careers in web development and AI-driven retail technology.

DRAWBACK OF THE EXISTING SYSTEM

Traditional e-commerce platforms have several limitations that frustrate customers and reduce sales:

1. No Personalized Product Discovery

Most platforms use basic keyword search without understanding user preferences. Customers waste time scrolling through irrelevant products.

2. Slow Customer Support

Traditional systems depend on human support staff who cannot provide 24/7 service, leading to long wait times for simple questions about sizing or materials.

3. Generic Recommendations

Existing recommendation systems show "frequently bought together" items without understanding individual style preferences or current trends.

4. Poor Mobile Experience

Many platforms are not optimized for mobile devices, resulting in slow loading, difficult navigation, and high cart abandonment on smartphones.

5. Security Concerns

Older systems lack modern encryption and secure authentication, putting customer payment data at risk.

6. No Real-Time Inventory Updates

Customers often discover items are out of stock only during checkout because inventory isn't synchronized in real-time.

7. Limited Business Analytics

Platforms provide basic sales reports but lack customer behavior insights, trend analysis, and data-driven recommendations for business decisions

8. Complicated Checkout Process

Long, confusing checkout flows with too many steps lead to cart abandonment and lost sales.

9. One-Size-Fits-All Interface

Every user sees the same homepage and product layout regardless of their preferences, history, or shopping behavior.

10. Difficult Product Management

As businesses grow, managing thousands of products becomes challenging without intelligent categorization and automated tools.

Summary:

Existing systems are functional but outdated. They lack intelligence, personalization, and modern features that today's customers expect. An AI-enhanced solution is essential to overcome these limitations.

SYSTEM REQUIREMENT SPECIFICATION

Hardware Requirements

Client Side (User's Device)

- **RAM:** Minimum 4GB
- **Browser:** Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- **Internet:** Minimum 2 Mbps
- **Display:** 1280x720 pixels minimum

Server Side

- **CPU:** Intel Core i5 or higher
- **RAM:** 16GB/16GB+
- **Storage:** 1TB SSD
- **OS:** Linux/Windows/MacOS
- **Web Server:** Gunicorn (for backend) + WhiteNoise (For static files)
- **Database:** PostgreSQL
- **Backend:** Django

Reason: Supports high read-write operations, fast API responses, and reliable storage.

Software Requirements

Frontend: Django templates + tailwind CSS

Backend: Django

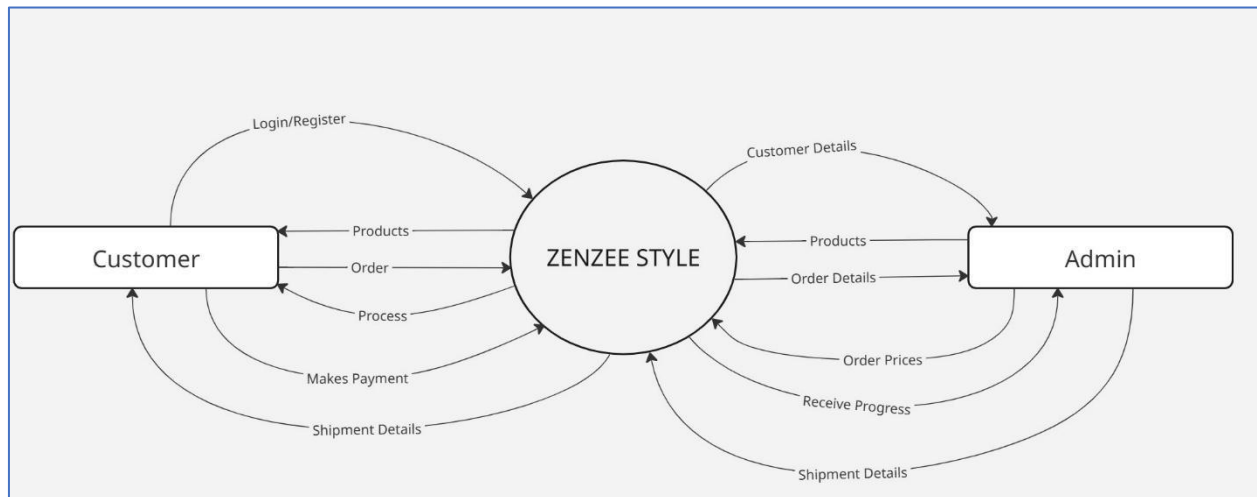
Payment Gateway: Razorpay

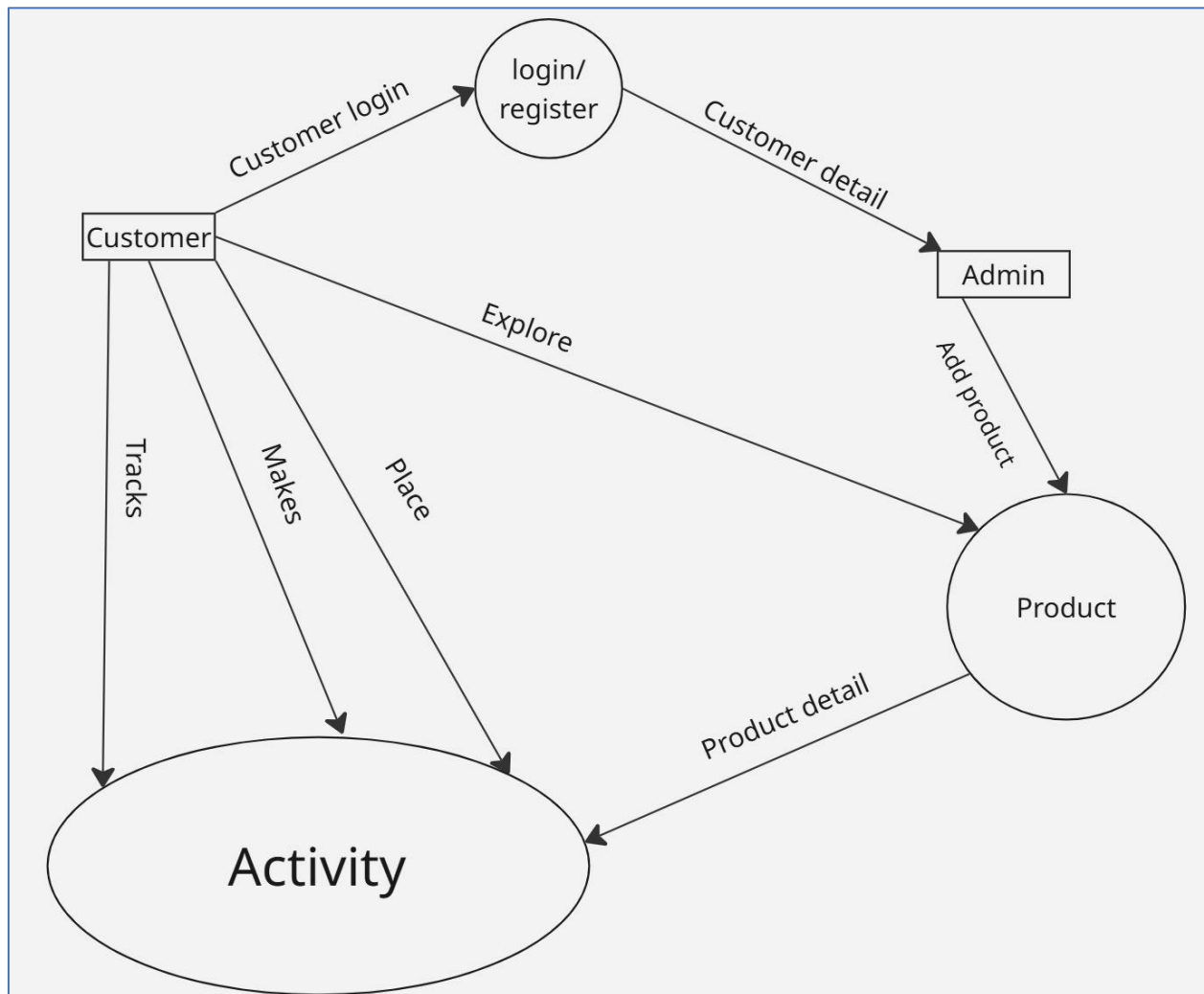
Authentication: Django-allauth

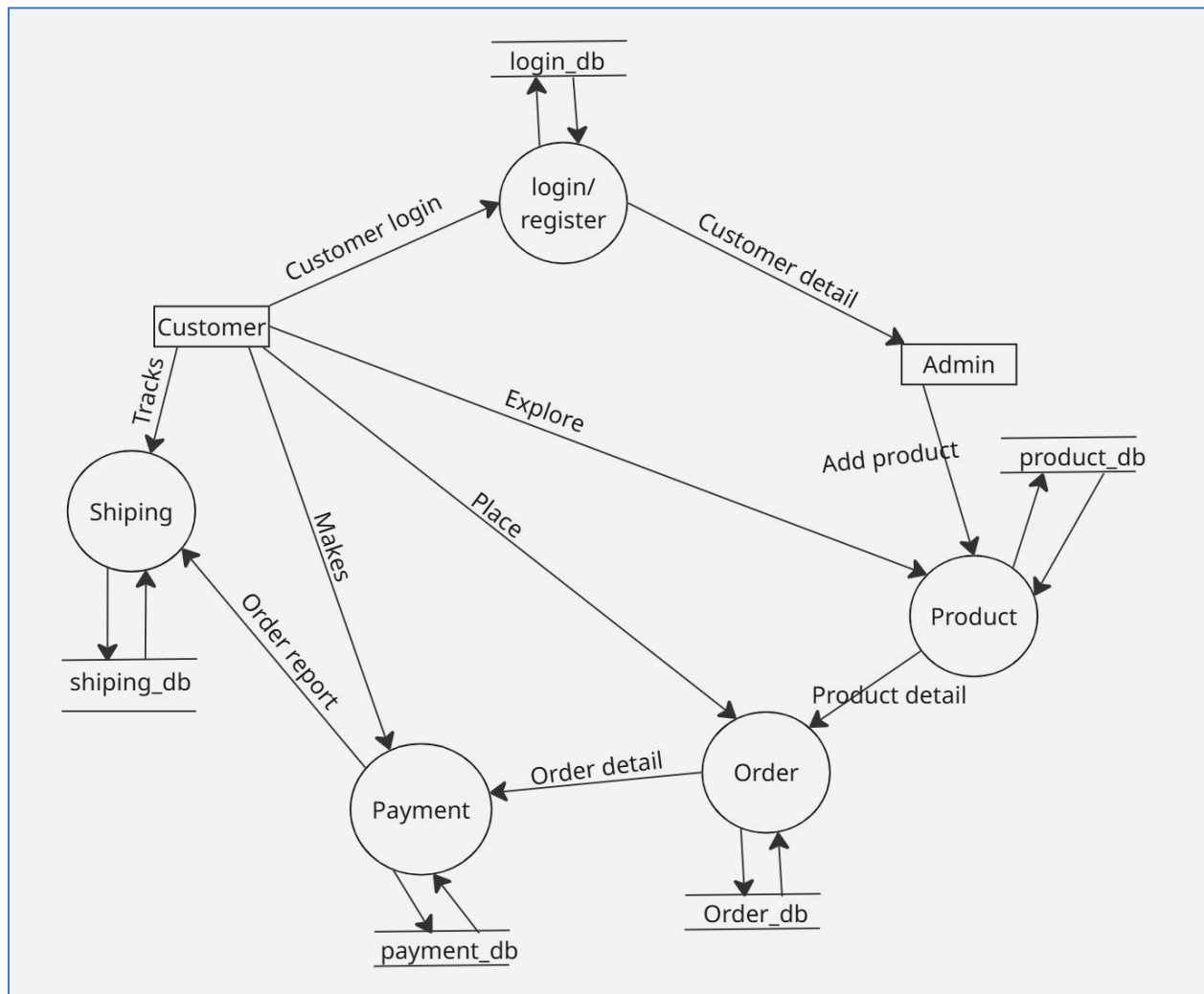
AI Chatbot: Custom NLP Model

DATA FLOW DIAGRAM

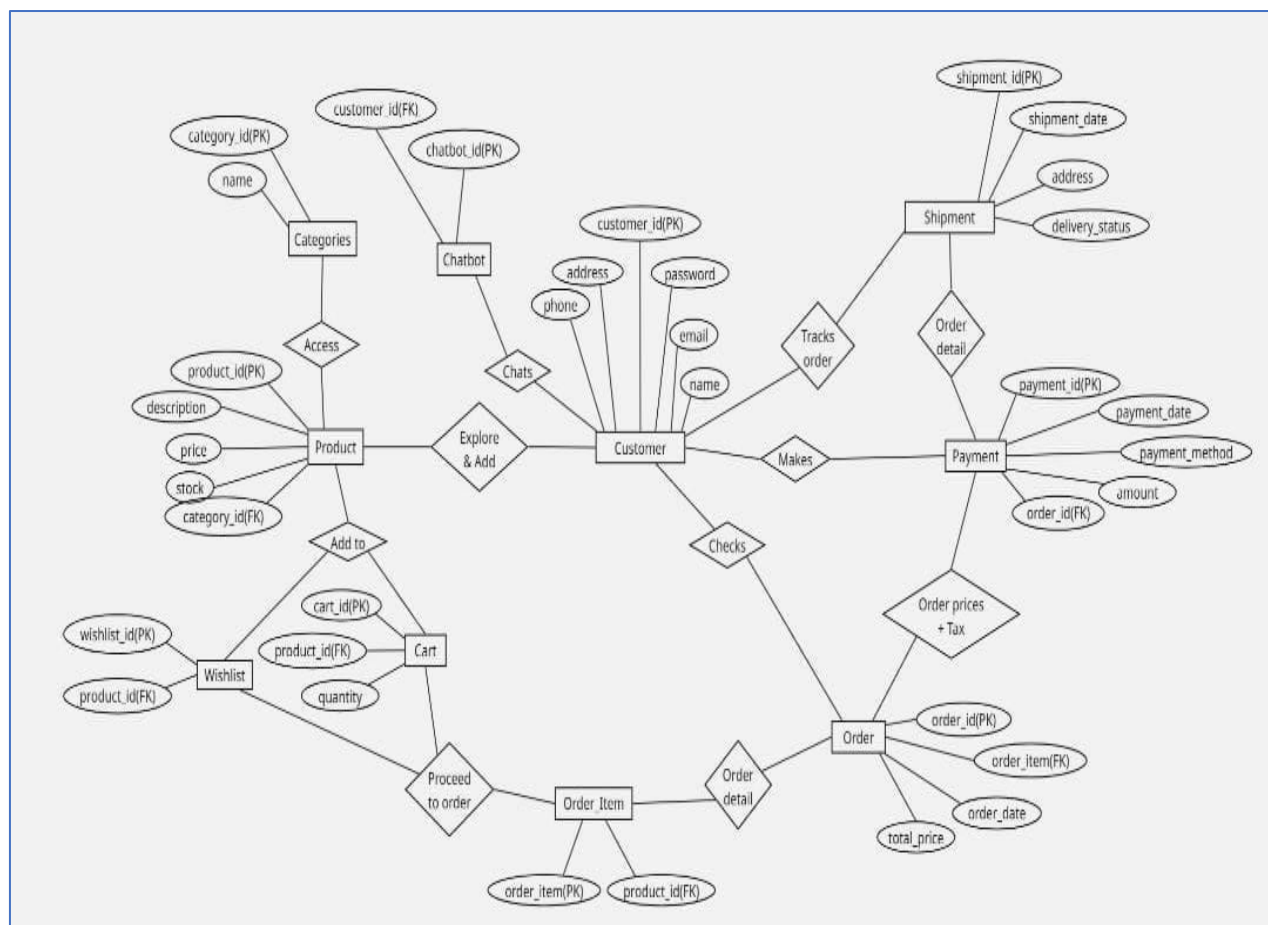
LEVEL 0 DFD:



LEVEL 1 DFD:

LEVEL 2 DFD:

ENTITY RELATIONSHIP DIAGRAM



FLOWCHART

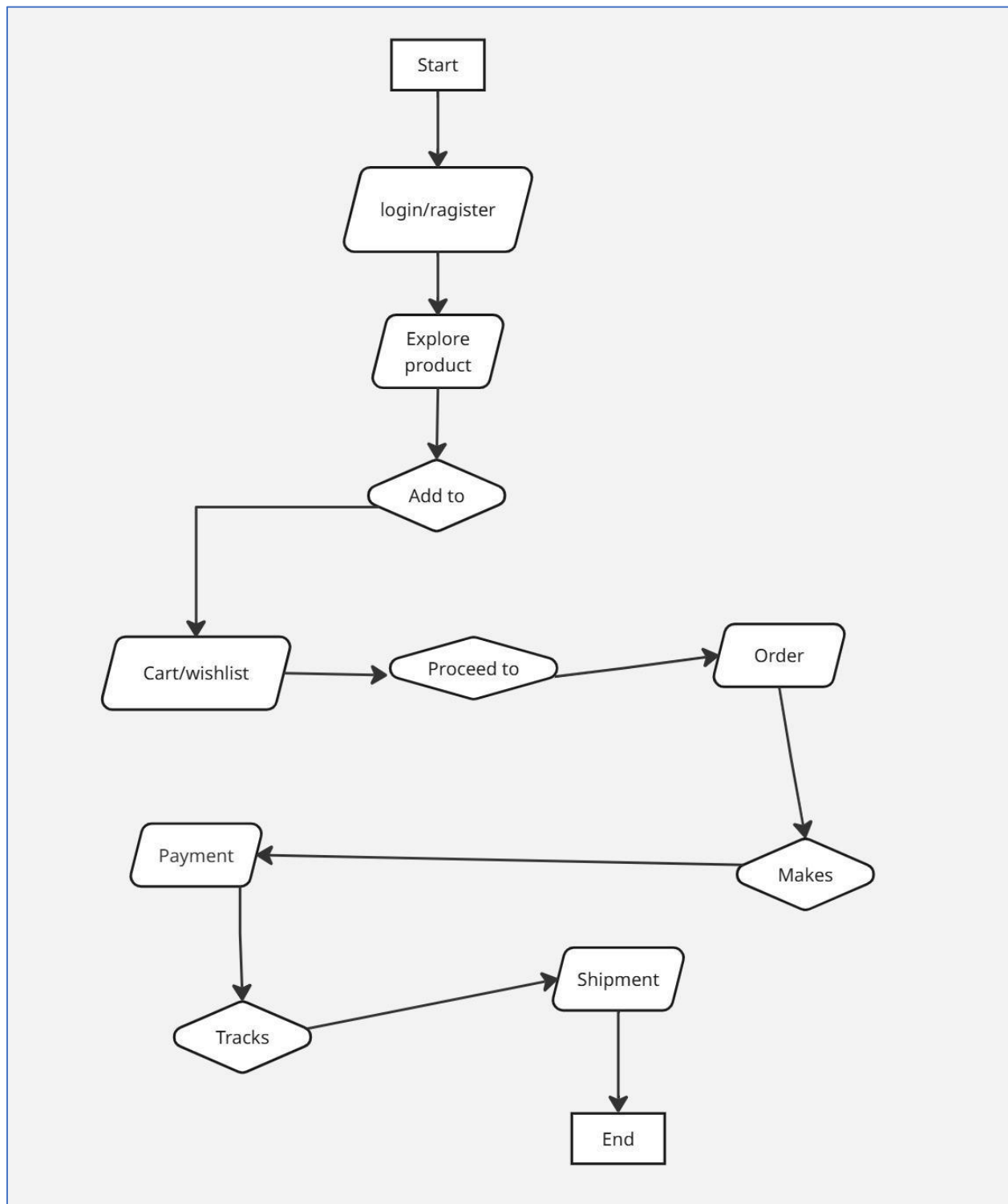


TABLE ARCHITECTURE

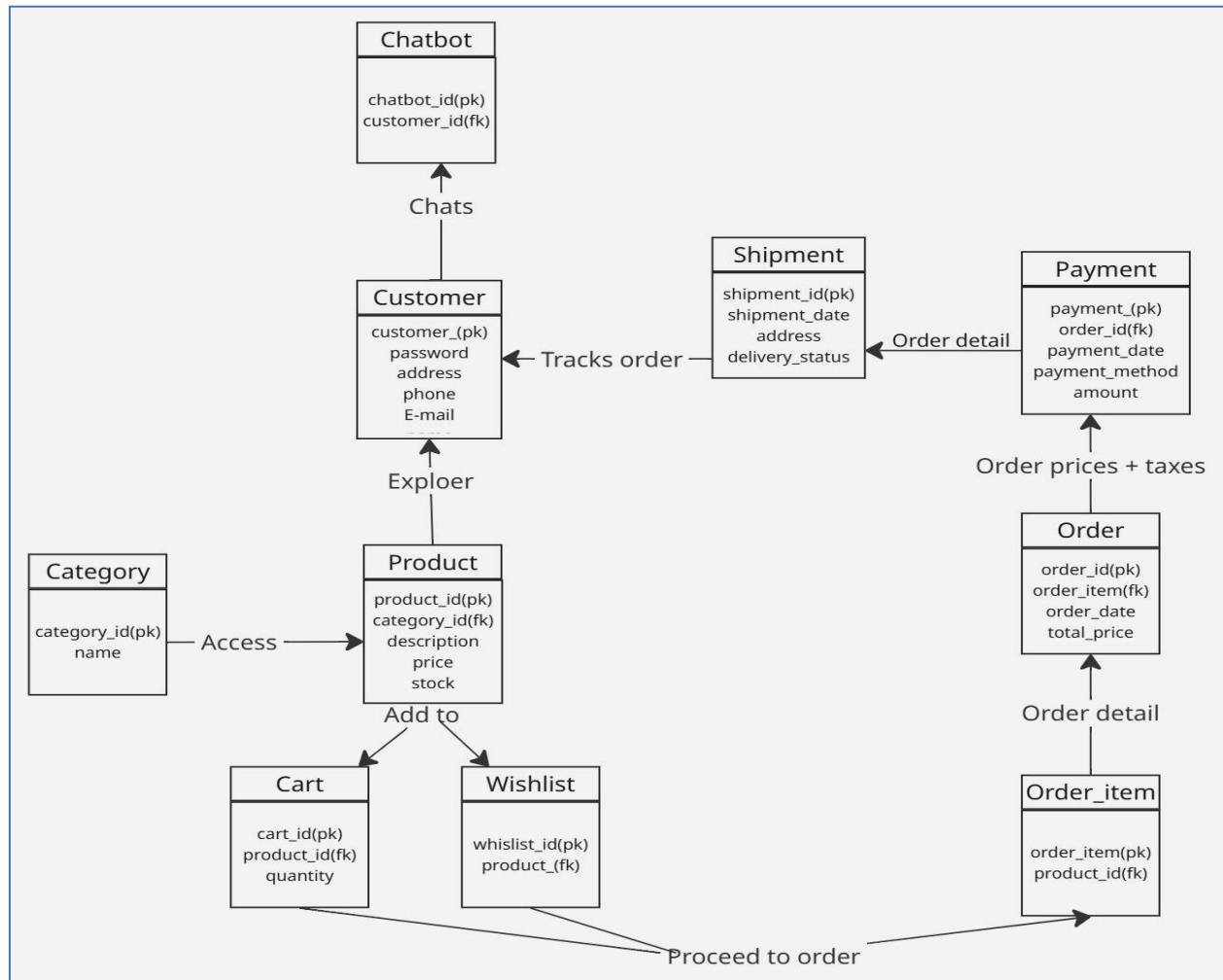
Costumer Table

Column	Data type	Constraints	Description
customer_id	INT	Primary Key, Auto Increment	Unique identifier for each customer
full_name	VARCHAR(100)	NOT NULL	Full name of the customer
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for login
phone	VARCHAR(15)	UNIQUE	Customer mobile number
password	VARCHAR(255)	NOT NULL	Encrypted password
address	TEXT	NULL	Customer delivery address
role	VARCHAR(20)	DEFAULT 'Customer'	User role (Customer/Admin)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation date

PAYMENT TABLE

Column Name	Data Type	Constraints	Description	
payment_id	INT	Primary Key, Auto Increment	Unique payment identifier	
customer_id	INT	Foreign Key	References Customer(customer_id)	
order_id	INT	NOT NULL	Associated order reference	
payment_method	VARCHAR(50)	NOT NULL	Mode of payment (UPI, Card, Wallet)	
payment_status	VARCHAR(30)	NOT NULL	Payment result (Success / Failed / Pending)	
transaction_id	VARCHAR(100)	UNIQUE	Gateway transaction reference	
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of payment	

CLASS DIAGRAM



MODULE DESCRIPTION

1. User Management Module

Purpose: Handles user registration, login, and profile management.

Features:

- User registration with email/phone
- Secure login with password hashing
- Role-based access (Customer/Admin)
- Profile updates
- Password reset
- Order history viewing

2. Product Management Module

Purpose: Manages product catalog and inventory.

Features:

- Admin can add/update/delete products
- Product categorization
- Image upload
- Stock management
- Product search and filtering
- Rating and reviews

3. Shopping Cart Module Purpose:

Manages cart operations.

Features:

- Add/remove items
- Update quantities
- Calculate totals

- Apply discount codes
- Persistent cart across sessions

6. Wishlist Module

Purpose: Allows users to save products for later.

Features:

- Add/remove from wishlist
- Move items to cart
- Price drop notifications (future)

7. Order Management Module

Purpose: Handles order lifecycle.

Features:

- Order creation
- Status tracking
- Order history
- Invoice generation
- Cancellation (within time limit)

8. Payment Integration Module

Purpose: Secure payment processing.

Features:

- Integration with Razorpay/Stripe
- Multiple payment methods
- Transaction verification
- Payment receipts

9. Search and Filter Module

Purpose: Powerful product search.

Features:

- Keyword search
- Auto-complete
- Filter by category, price, brand
- Sort by price, popularity, rating

10. Admin Dashboard Module Purpose:

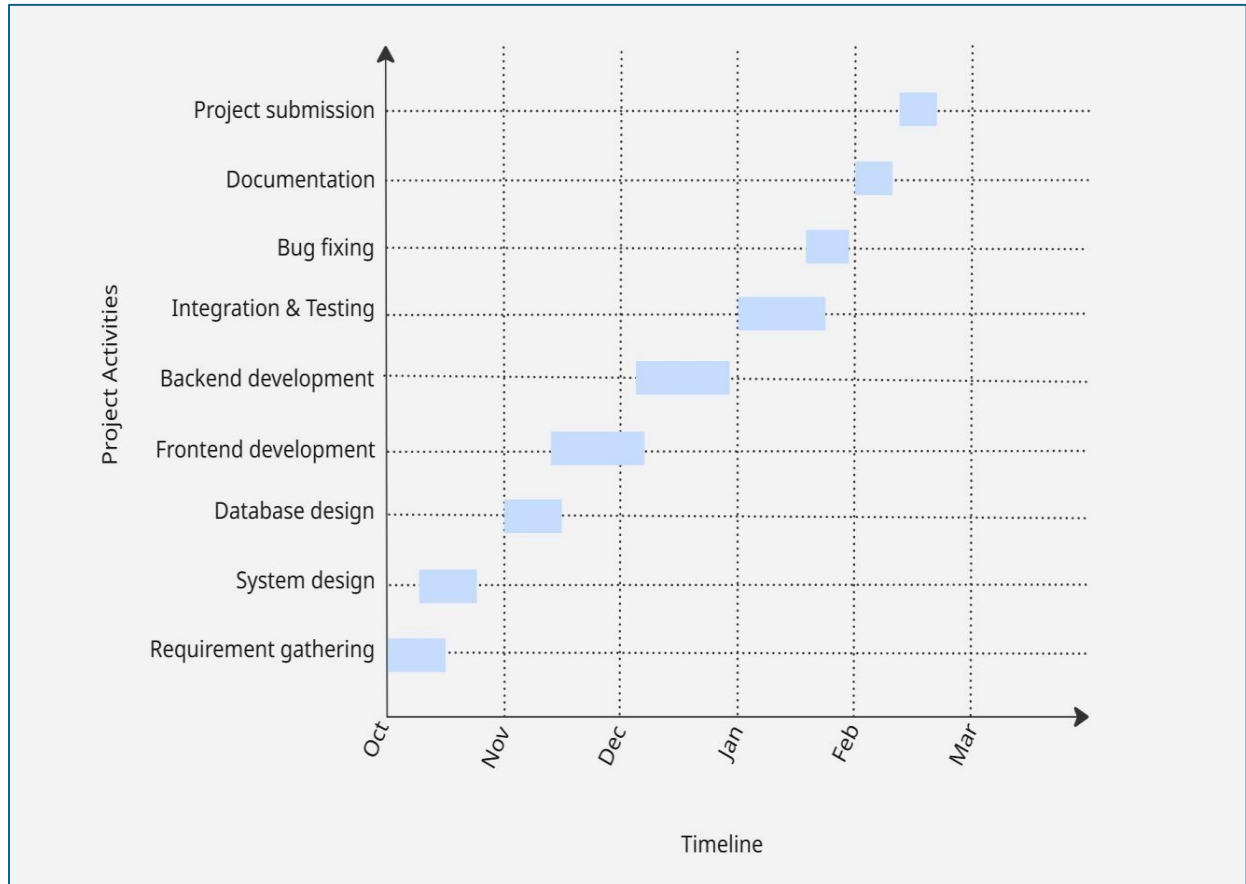
Administrative control panel.

Features:

- Sales analytics
- Product management
- User management
- Order management

Inventory tracking

GANTT CHART



TESTING OF THE PROJECT

1. Unit Testing

Test individual functions like login, add to cart, payment processing.

2. Integration Testing

Test module interactions: frontend-backend, database operations, payment gateway.

3. Functional Testing

- User registration and login
- Product search and filtering
- Cart and checkout
- Order placement
- Chatbot responses

4. Security Testing

- SQL injection prevention
- XSS protection
- Password encryption
- Secure payment processing

5. Performance Testing

- Load testing with multiple users
- Response time measurement
- Database query optimization

Implementation Of Security

Ensuring the security of user data, course content, and administrative controls is essential for the success and trustworthiness of an online course management system like Shiksha Path. The following outlines the key security mechanisms implemented in this project:

1. Authentication and Authorization

Secure Login System using encrypted passwords.

Only registered users (students/admins) can access their respective dashboards.

Login attempts are logged and rate-limited to prevent brute-force attacks.

Authorization:

Role-based access control (RBAC) ensures that users can only access functionalities permitted to their roles (e.g., admin, student).

Admins have elevated privileges such as course creation, content management, and user control.

2. Password Security

- HTTPS encryption
- Input validation and sanitization
- SQL injection prevention
- XSS protection
- Passwords are hashed using bcrypt or SHA-256 before storing in the database.
- Plain text passwords are never stored or transmitted.
- Strong password policy enforced (minimum length, character types).

3. Data Validation and Sanitization

All inputs (registration forms, course uploads, chatbots, etc.) are sanitized to avoid injection attacks SQL Injection prevention using parameterized queries or ORM techniques.

Cross-Site Scripting (XSS) prevention by escaping user-generated content.

4. Access Control

- Role-based permissions (customer/admin)
- Protected API endpoints
- Secure admin panel

5. Database Security

- Encrypted database connections
- Regular backups
- Access restrictions

FUTURE SCOPE

1. Mobile Application

- Native Android and iOS apps
- Push notifications
- Better mobile experience

2. Virtual Trial Room

- **AI-powered image processing to let customers upload photos and virtually try products**
- Computer vision integration
- Product overlay on user images
- Real-time visualization

3. Advanced AI Features

- Voice shopping assistant
- Visual product search
- Predictive analytics
- Dynamic pricing

4. AR Integration

- 3D product visualization
- AR try-on using camera
- Virtual showroom

5. Social Commerce

- Social media integration
- Share products
- Influencer partnerships
- Live shopping events

6. Enhanced Analytics

- Customer behavior prediction
- Sales forecasting
- Advanced reporting

7. Multi-language Support

- Regional languages
- Multiple currencies
- International shipping

LIMITATIONS OF THE PROJECT

1. Basic AI Implementation

- Simple recommendation algorithm
- Rule-based chatbot with limited intelligence
- Requires more user data for better accuracy

2. No Mobile App

- Web-based only
- No native mobile applications
- Limited offline functionality

3. Basic Chatbot

- Cannot handle very complex queries
- Requires training with more data
- No emotion detection

4. Limited Payment Options

- Only 1-2 payment gateways
- No cryptocurrency support
- No EMI options

5. No Multi-language

- English only interface
- Limits accessibility for regional users

6. Virtual Trial Room Not Included

- Current version doesn't have image-based try-on
- Planned for future enhancement
- Requires advanced computer vision skills

7. Basic Analytics

- Simple dashboard metrics
- No predictive analytics
- Limited business intelligence features

8. Security Audit Pending

- Basic security implemented
- Professional security audit required
- Penetration testing not performed

9. Limited Third-Party Integration

- Basic payment gateway only
- No logistics API integration
- No CRM connection

10. Testing Coverage

- Manual testing performed
- Automated testing limited
- Performance testing in development environment only

Conclusion:

These limitations reflect the current scope as a student project. The platform provides a strong foundation that can be enhanced with more time, resources, and advanced technical skills.

BIBLIOGRAPHY

Books & Study Material:

- Software Engineering by Roger S. Pressman – For understanding SDLC, Feasibility Study, and Testing strategies. (
https://openlibrary.org/books/OL9263535M/Software_Engineering?utm_source=chatgpt.com)
- Database System Concepts by Abraham Silberschatz – For RDBMS design and normalization.
(https://openlibrary.org/books/OL22462529M/Database_system_concepts?utm_source=chatgpt.com)
- Web Technologies by Uttam K. Roy – For understanding the integration of frontend and backend technologies.
(https://openlibrary.org/books/OL28754748M/Web_Technologies?utm_source=chatgpt.com)

Research Articles and Educational Platforms:

- Tutorials Point and GeeksforGeeks – For conceptual understanding and sample code references.
- Medium.com – For insights and community-shared best practices in online course systems.
- Coursera, edX, and Udemy – For analysing existing platforms and gathering ideas for features and enhancements.

Technical Support & Discussions:

- Guide – For solving development-related queries and logic implementation.
- GitHub Repositories – For observing open-source LMS project structures and module inspiration.

Acknowledgments:

This project is developed as part of the BCA program at IGNOU. Special thanks to the project guide, faculty members, and peers for their support and feedback throughout the development process.

All external resources, libraries, and APIs used are properly attributed and their licenses are followed.

Project Report

Context

1. Synopsis	3
2. Introduction	33
3. Objective	34
4. Software Requirements Specification	36
5. System Requirements	42
6. Analysis of Project	43
6.1 Data Flow Diagram	
6.2 Entity Relationship Diagram	
6.3 Table Architecture	
6.4 Class Diagram	
6.5 Flowchart	
6.6 Gant chart	
7. Software Design Document	52
8. Program Code	58
9. Testing	119
10. Input and Output Screens	125
11. COCOMO Model	132
12. Cost Estimation	134
13. Implementation Of Security	136
14. Limitations Of the Project	138
15. Future Application	140
16. Bibliography	142

INTRODUCTION

The rapid growth of information technology and internet services has transformed the way people purchase goods and services. E-commerce platforms have become an essential part of modern business by enabling users to buy products online with convenience and efficiency. However, many existing e-commerce systems still lack interactive support, personalized assistance, and intelligent user engagement, which often results in poor user experience and customer dissatisfaction.

ZEN-ZEE is a web-based e-commerce application developed to address these limitations by providing a modern and interactive online shopping platform. The system integrates essential ecommerce functionalities such as user registration, product browsing, shopping cart management, secure payment processing, and order tracking. In addition to these standard features, ZEN-ZEE includes **AI-assisted components**, mainly in the form of an integrated chatbot and rule-based recommendation support, to enhance user interaction.

The AI component in ZEN-ZEE is not designed as a complex artificial intelligence model but rather as an **AI-integrated assistance system**. The chatbot helps users by answering product related queries, guiding them during navigation, and providing real-time assistance during the shopping process. This approach improves usability while keeping the system realistic and feasible within the academic scope of a BCA project.

The system is designed using a structured and modular architecture, which is clearly represented through system diagrams such as Data Flow Diagrams (DFD), Entity Relationship Diagrams (ERD), table architecture, and module diagrams. These diagrams explain how data flows within the system and how different components interact with each other.

The primary purpose of the ZEN-ZEE project is to design and implement a reliable, secure, and user-friendly e-commerce platform with AI-assisted interaction, demonstrating practical knowledge of web development, database management, and system design.

OBJECTIVE OF PROJECT

The main objective is to create an intelligent e-commerce platform that uses AI to enhance product discovery and customer support. The specific goals are:

1. To Provide Intelligent Product Discovery

Implement an AI-powered recommendation system that analyzes user preferences, browsing patterns, and trending products to automatically suggest relevant items. This saves time and helps customers find what they need quickly.

2. To Enable Real-Time Customer Support Through AI Chatbot

Integrate a contextual chatbot that understands customer queries in natural language and provides instant help with products, sizing, materials, and purchasing guidance 24/7.

3. To Facilitate Personalized Shopping Experiences

Analyze user behavior, preferences, style choices, and budget to deliver highly personalized product recommendations tailored to each customer.

4. To Offer Comprehensive Product Management

Organize products into clear categories (clothing, footwear, accessories, electronics) with advanced filtering by price, brand, color, size, and ratings.

5. To Implement Smart Cart and Wishlist Features

Allow users to add products to cart, save items for later in a Wishlist, and receive notifications about price drops or restocking.

6. To Integrate Secure Payment and Order Management

Support multiple payment methods (cards, UPI, wallets) with secure processing, automatic order tracking, invoice generation, and delivery updates.

7. To Provide Admin Dashboard for Business Management

Give administrators tools to manage products, view sales analytics, track customer behavior, monitor inventory, and make data-driven decisions.

8. To Ensure Data Security and Privacy

Protect user information using encryption, secure authentication, and role-based access control. Comply with data protection standards.

9. To Build a Scalable System for Future Growth

Design the architecture to support future features like mobile apps, advanced analytics, and integration with other services.

Conclusion:

These objectives aim to make the platform comprehensive, user-friendly, and future-ready. By combining AI with good design, the platform will improve the online shopping experience.

Software Requirements Specification

Zen-Zee Style – E-Commerce Web Application

1. Introduction

1.1 Purpose

This document provides a detailed description of the requirements for the Zen-Zee Style e-commerce web application. It defines the system's functional and non-functional requirements and serves as a reference for development, testing, and deployment.

1.2 Scope

Zen-Zee Style is a web-based e-commerce platform that allows users to browse and purchase products, while enabling sellers to manage and sell their products. The system includes authentication, product management, cart functionality, order processing, payment integration, and order tracking.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS** – Software Requirements Specification
- **UI** – User Interface
- **DB** – Database
- **OAuth** – Open Authorization
- **API** – Application Programming Interface

1.4 Overview

This document outlines the system's overall description, system features, external interfaces, and constraints. It serves as a blueprint for the application.

2. Overall Description

2.1 Product Perspective

Zen-Zee Style is a standalone web application developed using Django. It interacts with:

- PostgreSQL database for data storage
- GitHub OAuth for authentication

- Razorpay API for payment processing

2.2 Product Functions

The system provides the following major functionalities:

- User authentication via GitHub
- Role-based access (Customer/Seller)
- Product browsing and management
- Cart and checkout system
- Online payment processing
- Order tracking and management

2.3 User Classes and Characteristics

Customer

- Can browse products
- Add items to cart
- Place orders
- Track order status

Seller

- Can manage products (add/edit/delete)
- View orders related to their products
- Update order delivery status

Admin

- Full system access via Django admin panel
- Manages users, products, and orders

2.4 Operating Environment

- Web browser (Chrome, Edge, Firefox)
- Backend server: Django
- Database: PostgreSQL (Render Cloud)

- Deployment platform: Render

2.5 Design and Implementation Constraints

- Must use Django framework
- Authentication restricted to GitHub OAuth
- Payment handled via Razorpay
- Deployment on Render platform
- Limited free-tier database availability

2.6 Assumptions and Dependencies

- Users have a GitHub account
- Internet connection is available
- Razorpay services are operational
- Render services are available

3. System Features

3.1 User Authentication

- Users can log in using GitHub OAuth
- Secure authentication handled by django-allauth

3.2 User Role Management

- Users select role (Customer/Seller) during onboarding
- Users can switch roles from profile settings
- System restricts access based on role

3.3 Product Management

- Sellers can add, edit, and delete products
- Each product includes name, price, description, image, and category

3.4 Search Functionality

- Users can search products by name or category
- Search results update dynamically

3.5 Cart Management

- Users can add/remove/update items in cart
- Cart maintains session data
- Displays total price

3.6 Order Management

- Users can place orders from cart
- Orders store user, product details, and total amount

3.7 Order Tracking

- Order status stages:
 - Pending
 - Confirmed
 - Shipped
 - Out for Delivery
 - Delivered
- Users can view real-time status

3.8 Payment System

- Razorpay integration for online payments
- Secure transaction processing
- Payment status validation

4. External Interface Requirements

4.1 User Interface

- Web-based UI using HTML and Tailwind CSS
- Responsive design for multiple devices

4.2 Hardware Interfaces

- No special hardware requirements

4.3 Software Interfaces

- GitHub OAuth API
- Razorpay Payment API
- PostgreSQL Database

4.4 Communication Interfaces

- HTTP/HTTPS protocol for client-server communication

5. Non-Functional Requirements

5.1 Performance Requirements

- System should handle multiple users simultaneously
- Fast page load times

5.2 Security Requirements

- Secure authentication using OAuth
- Protection of user data and payment information
- Use of HTTPS

5.3 Reliability

- System should be available with minimal downtime
- Proper error handling mechanisms

5.4 Usability

- User-friendly interface
- Easy navigation and clear UI

5.5 Scalability

- System should support increasing number of users and products

6. Future Enhancements

- Product reviews and ratings
- Wishlist feature
- Email notifications

- Mobile application support
- Advanced seller analytics

7. Conclusion

The SRS document outlines the complete requirements for the Zen-Zee Style e-commerce platform. It provides a structured foundation for development, testing, and deployment, ensuring the system meets user and business needs.

SYSTEM REQUIREMENT

Hardware Requirements

Client Side (User's Device)

- **RAM:** Minimum 4GB
- **Browser:** Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- **Internet:** Minimum 2 Mbps
- **Display:** 1280x720 pixels minimum

Server Side

- **CPU:** Intel Core i5 or higher
- **RAM:** 16GB/16GB+
- **Storage:** 1TB SSD
- **OS:** Linux/Windows/MacOS
- **Web Server:** Gunicorn (for backend) + WhiteNoise (For static files)
- **Database:** PostgreSQL
- **Backend:** Django

Reason: Supports high read-write operations, fast API responses, and reliable storage.

Software Requirements

Frontend: Django templates + tailwind CSS

Backend: Django

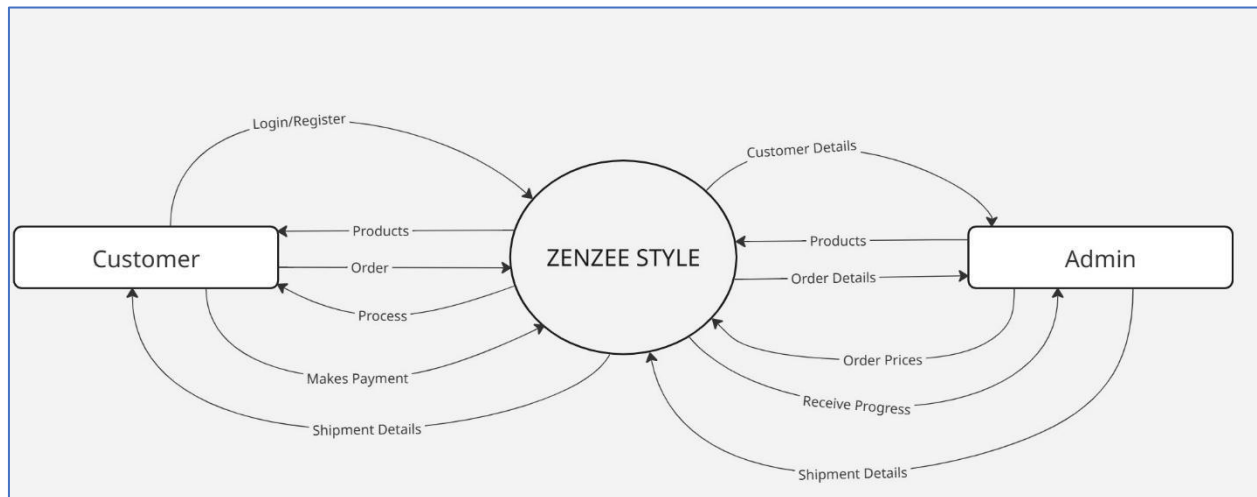
Payment Gateway: Razorpay

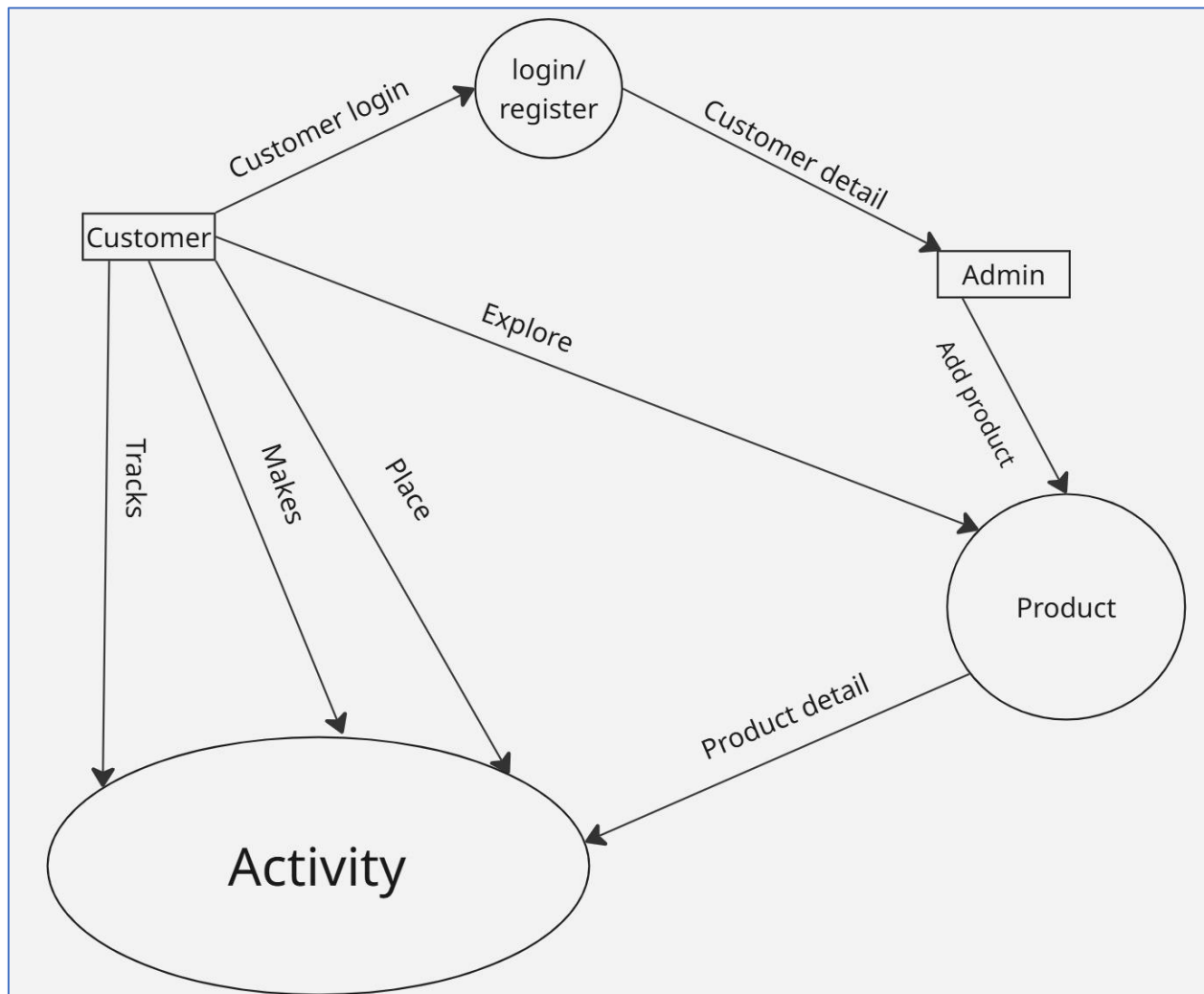
Authentication: Django-allauth

AI Chatbot: Custom NLP Model

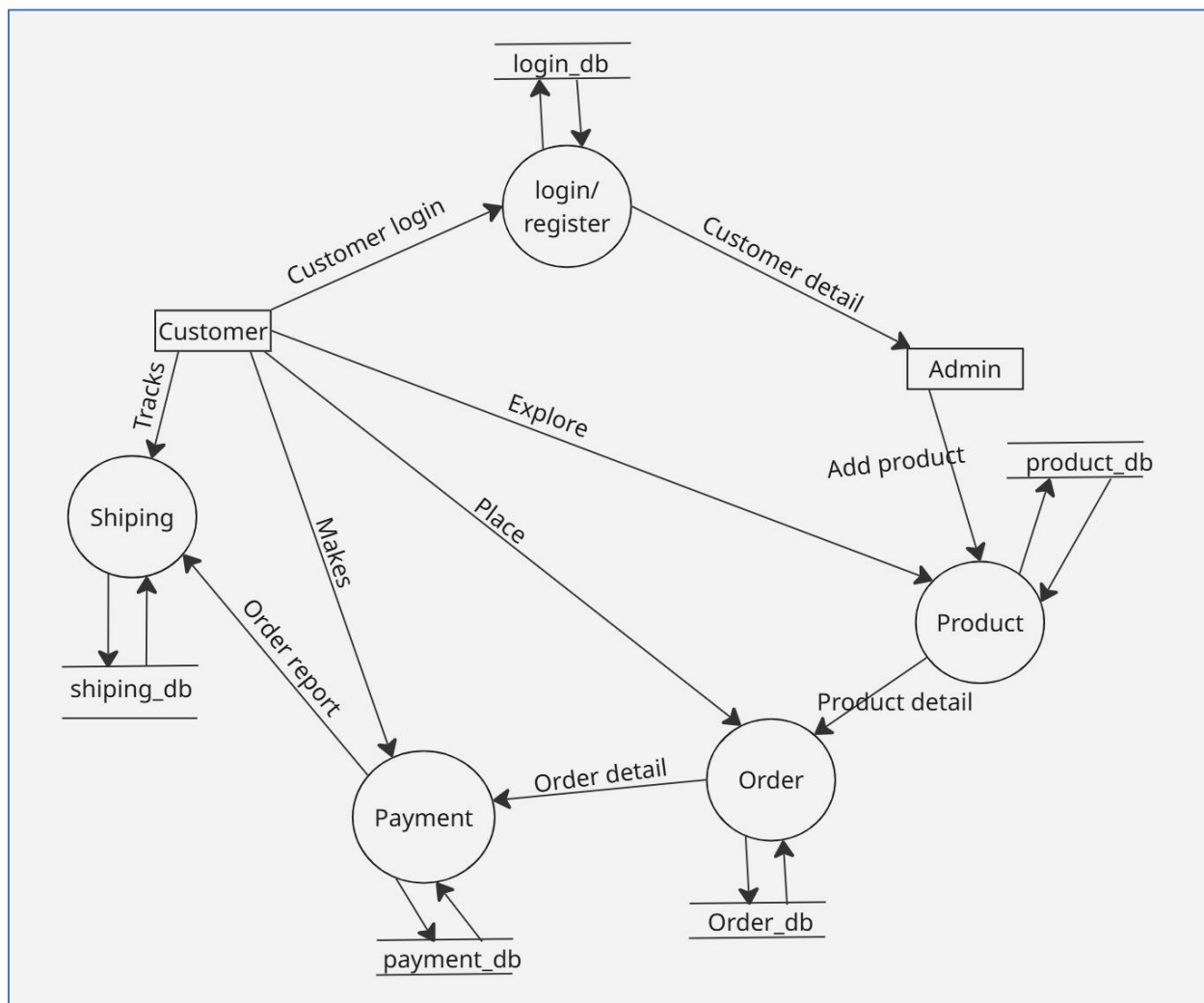
DATA FLOW DIAGRAM

LEVEL 0 DFD:

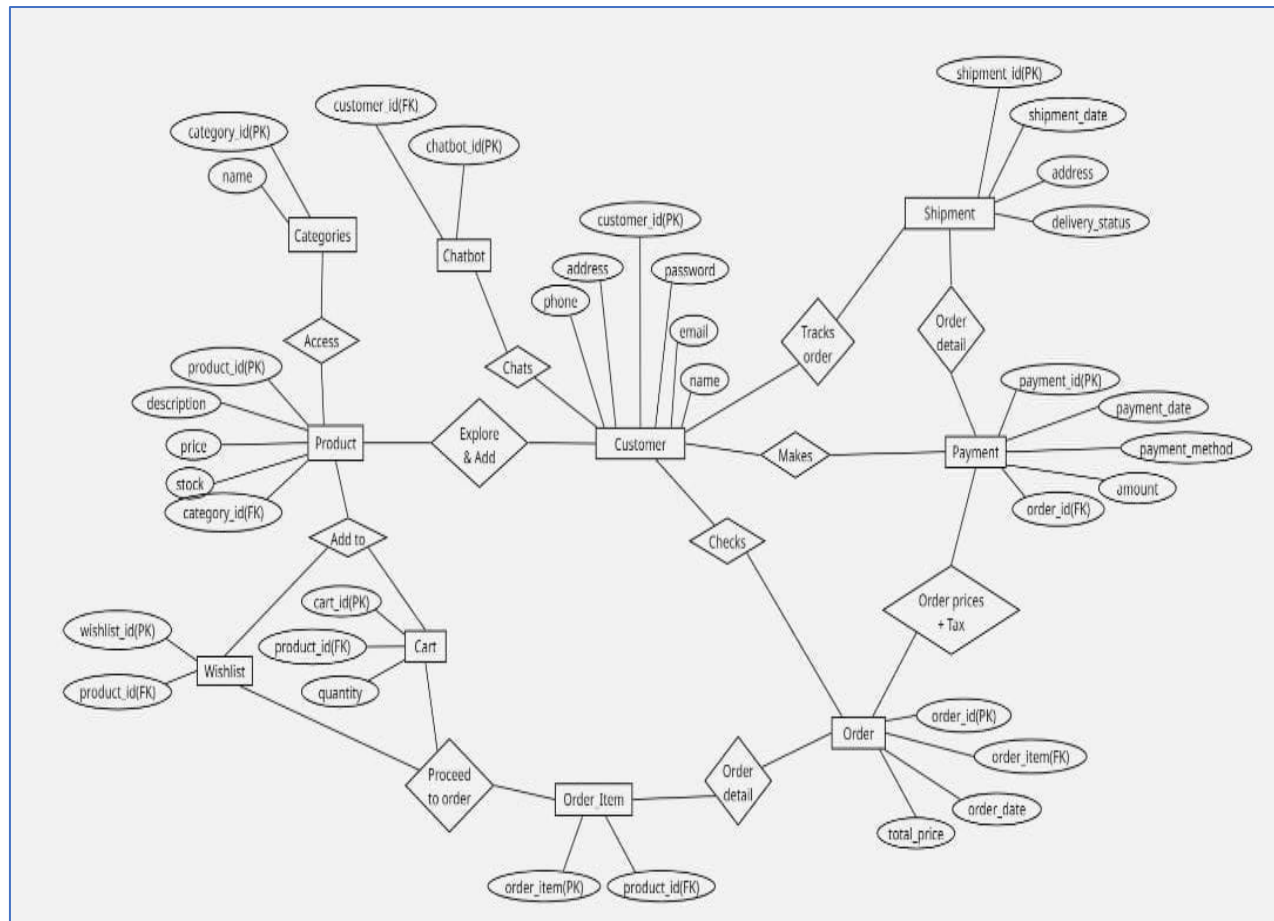


LEVEL 1 DFD:

LEVEL 2 DFD:



ENTITY RELATIONSHIP DIAGRAM



FLOWCHART

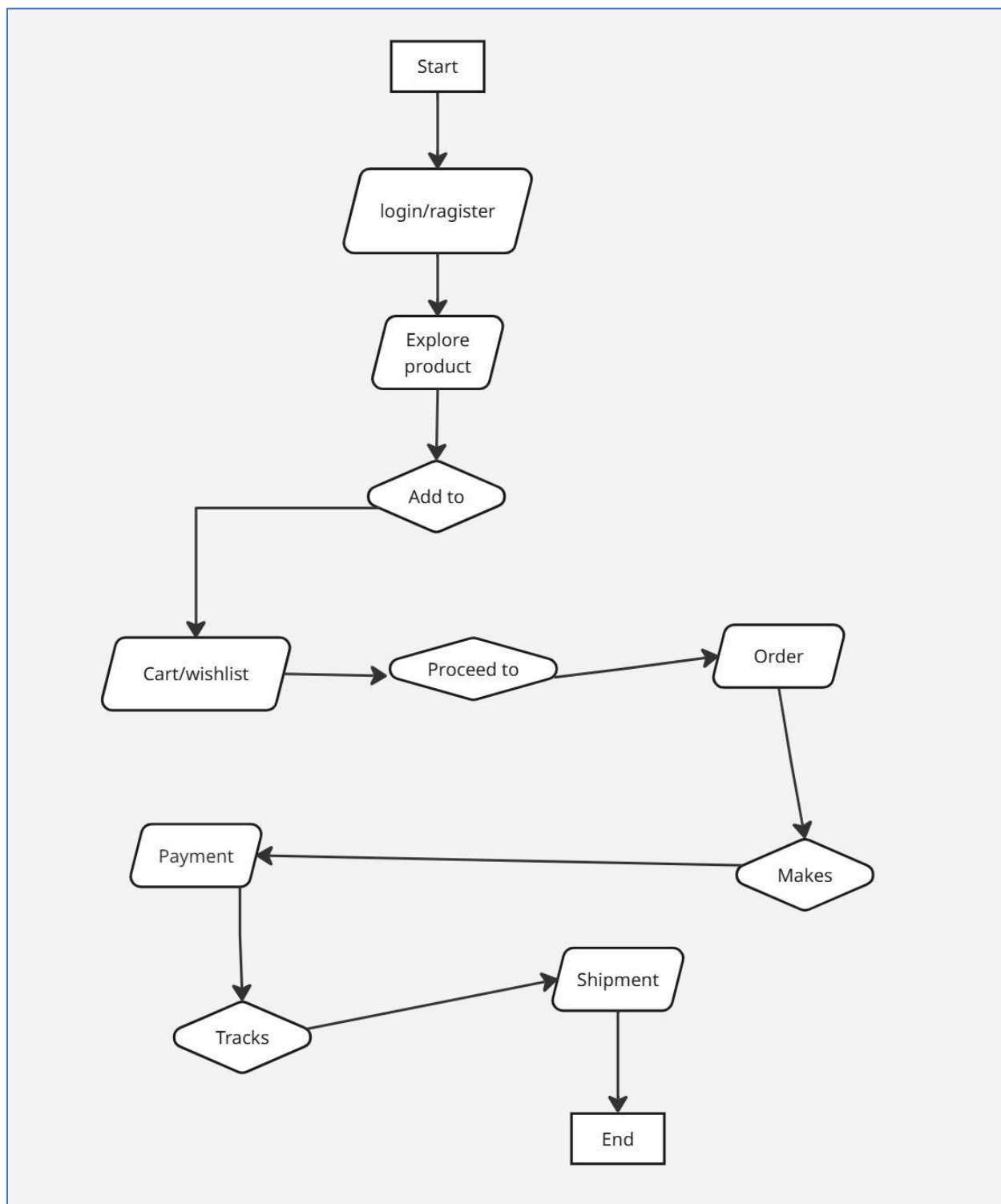


TABLE ARCHITECTURE

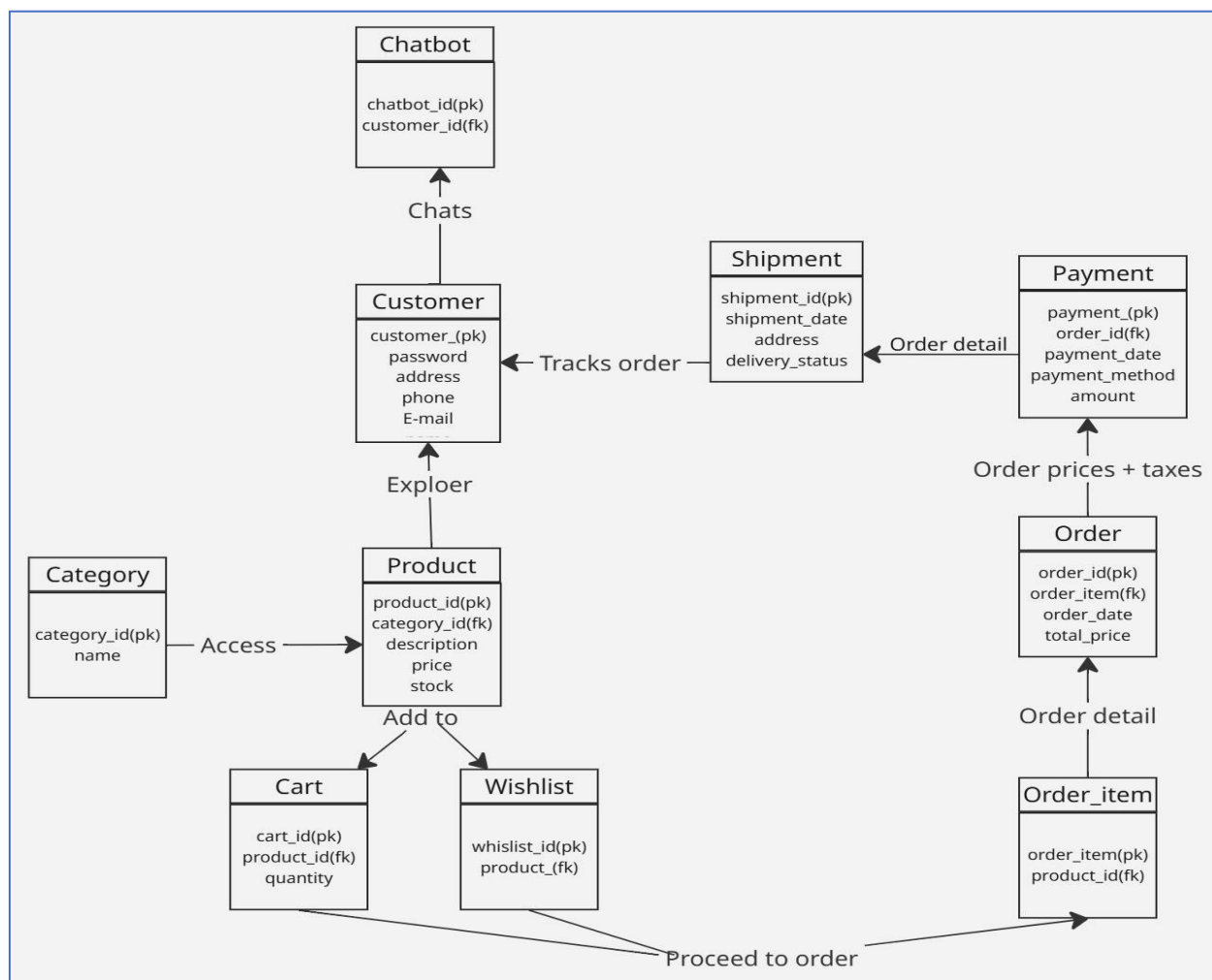
Costumer Table

Column	Data type	Constraints	Description
customer_id	INT	Primary Key, Auto Increment	Unique identifier for each customer
full_name	VARCHAR(100)	NOT NULL	Full name of the customer
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address used for login
phone	VARCHAR(15)	UNIQUE	Customer mobile number
password	VARCHAR(255)	NOT NULL	Encrypted password
address	TEXT	NULL	Customer delivery address
role	VARCHAR(20)	DEFAULT 'Customer'	User role (Customer/Admin)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation date

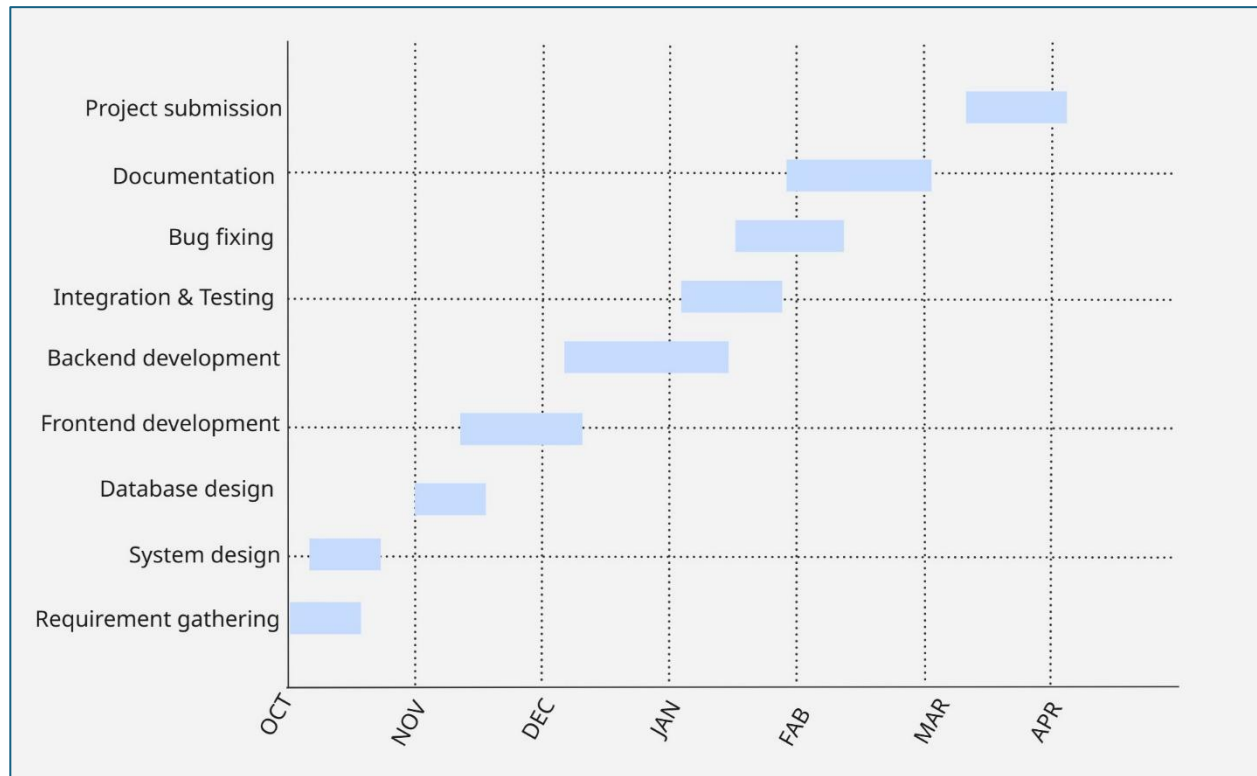
PAYMENT TABLE

Column Name	Data Type	Constraints	Description	
payment_id	INT	Primary Key, Auto Increment	Unique payment identifier	
customer_id	INT	Foreign Key	References Customer(customer_id)	
order_id	INT	NOT NULL	Associated order reference	
payment_method	VARCHAR(50)	NOT NULL	Mode of payment (UPI, Card, Wallet)	
payment_status	VARCHAR(30)	NOT NULL	Payment result (Success / Failed / Pending)	
transaction_id	VARCHAR(100)	UNIQUE	Gateway transaction reference	
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of payment	

CLASS DIAGRAM



Gant chart



Software Design Document

Zen-Zee Style – E-Commerce Web Application

1. Introduction

1.1 Purpose

This document provides a detailed design of the Zen-Zee Style e-commerce web application. It describes the system architecture, components, data design, and interaction between modules.

1.2 Scope

The system is designed to support online buying and selling of products with role-based access for customers and sellers. It includes authentication, product management, cart functionality, order processing, payment integration, and order tracking.

1.3 Definitions

- **MVT** – Model View Template
- **API** – Application Programming Interface
- **ORM** – Object Relational Mapping

2. System Architecture

The application follows the **Django MVT (Model-View-Template) architecture**:

- **Model Layer:** Handles database structure and data logic
- **View Layer:** Processes user requests and business logic
- **Template Layer:** Renders UI using HTML and Tailwind CSS

Architecture Flow:

User → URL Routing → View → Model → Template → Response

3. Module Design

3.1 User Module

- Handles authentication via GitHub OAuth (django-allauth)
- Manages user profiles and roles (Customer/Seller)

- Allows role switching

3.2 Product Module

- Manages product data
- Sellers can:
 - Add products
 - Edit products
 - Delete products
- Products linked to seller accounts

3.3 Cart Module

- Session-based cart implementation
- Stores product ID and quantity
- Calculates total price dynamically

3.4 Order Module

- Converts cart into order
- Stores:
 - User details
 - Product details
 - Total amount
 - Order status

3.5 Payment Module

- Integrates Razorpay API
- Handles:
 - Payment creation
 - Payment verification
 - Payment status update

3.6 Order Tracking Module

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

- Tracks order status:
 - Pending → Confirmed → Shipped → Out for Delivery → Delivered
- Sellers update status
- Users view progress

4. Data Design

4.1 Database Schema (Overview)

User & Profile

- User (Django default)
- Profile:
 - user (OneToOne)
 - role (Customer/Seller)

Product

- id
- name
- description
- price
- image
- stock
- category
- seller (ForeignKey to User)

Order

- id
- user
- total_amount
- status
- created_at

OrderItem

- order (ForeignKey)
- product (ForeignKey)
- quantity
- price

4.2 Relationships

- One User → Many Products (Seller)
- One User → Many Orders
- One Order → Many OrderItems
- One Product → Many OrderItems

5. Interface Design

5.1 User Interface

- Built using HTML and Tailwind CSS
- Responsive design
- Key pages:
 - Home (Product Listing)
 - Product Detail
 - Cart Page
 - Checkout Page
 - Seller Dashboard
 - Order Tracking Page

5.2 External Interfaces

- GitHub OAuth API for authentication
- Razorpay API for payments
- PostgreSQL database connection

6. Component Design

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

6.1 Views

- Handle request-response cycle
- Example:
 - Product list view
 - Cart view
 - Checkout view

6.2 Templates

- Reusable components:
 - base.html
 - navbar
 - product cards
- Dynamic rendering using Django template engine

6.3 URL Routing

- Maps URLs to views
- Example:
 - / → Home
 - /product/<id>/ → Product Detail
 - /cart/ → Cart Page

7. Security Design

- Authentication via GitHub OAuth
- Role-based access control
- Secure payment handling via Razorpay
- Environment variables for sensitive data

8. Deployment Design

- Hosted on Render
- PostgreSQL database on Render

- Gunicorn as application server
- Whitenoise for static files
- Environment variables for configuration

9. Error Handling

- Try-except blocks for database operations
- User-friendly error messages
- Payment failure handling

10. Future Design Enhancements

- Microservices architecture
- API-based backend (Django REST Framework)
- Mobile app integration
- Advanced analytics dashboard

11. Conclusion

The Software Design Document provides a structured overview of the system architecture and design. It ensures clarity in implementation and helps maintain scalability, security, and efficiency of the Zen-Zee Style e-commerce application.

PROGRAM CODE

Zen-Zee_style/cart:

cart/apps.py

```
from django.apps import AppConfig
```

```
class CartConfig(AppConfig):
```

```
    default_auto_field = "django.db.models.BigAutoField"
```

```
    name = "cart"
```

```
    verbose_name = "Cart"
```

cart/context_processors.py

```
from .utils import get_cart_items
```

```
def cart_summary(request):
```

```
    _, total = get_cart_items(request.session)
```

```
    return {"cart_total_amount": total}
```

cart/urls.py

```
from django.urls import path
```

```
from . import views
```

```
app_name = "cart"
```

```
urlpatterns = [
    path("", views.cart_detail, name="cart_detail"),
    path("add/<int:product_id>/", views.add_to_cart, name="add_to_cart"),
    path("remove/<int:product_id>/", views.remove_from_cart, name="remove_from_cart"),
    path("update/<int:product_id>/", views.update_cart, name="update_cart"),
]
```

cart/utils.py

```
from django.shortcuts import get_object_or_404
from products.models import Product
```

```
CART_SESSION_KEY = "cart"
```

```
def _load_cart(session):
    return session.get(CART_SESSION_KEY, {})
```

```
def _save_cart(session, cart):
    session[CART_SESSION_KEY] = cart
    session.modified = True
```

```
def add_item(session, product_id, quantity=1):
    cart = _load_cart(session)
```

```

product = get_object_or_404(Product, pk=product_id)
item = cart.get(str(product_id), {"quantity": 0, "price": str(product.price)})
item["quantity"] = item.get("quantity", 0) + quantity
cart[str(product_id)] = item
_save_cart(session, cart)

```

```

def remove_item(session, product_id):
    cart = _load_cart(session)
    cart.pop(str(product_id), None)
    _save_cart(session, cart)

```

```

def update_item(session, product_id, quantity):
    cart = _load_cart(session)
    if str(product_id) in cart:
        if quantity <= 0:
            cart.pop(str(product_id), None)
        else:
            cart[str(product_id)]["quantity"] = quantity
    _save_cart(session, cart)

```

```

def get_cart_items(session):
    cart = _load_cart(session)
    products = Product.objects.filter(id__in=[int(pk) for pk in cart.keys()])

```

```

product_map = {str(product.id): product for product in products}
items = []
total = 0
for product_id, data in cart.items():
    product = product_map.get(product_id)
    if not product:
        continue
    quantity = data.get("quantity", 0)
    subtotal = product.price * quantity
    total += subtotal
    items.append({"product": product, "quantity": quantity, "subtotal": subtotal})
return items, total

```

```

def clear_cart(session):
    if CART_SESSION_KEY in session:
        del session[CART_SESSION_KEY]
        session.modified = True

```

cart/views.py

```

from django.shortcuts import redirect, render
from django.views.decorators.http import require_POST
from .utils import add_item, remove_item, update_item, get_cart_items

```

```

def cart_detail(request):

```

```
items, total = get_cart_items(request.session)

return render(request, "cart/cart_detail.html", {"items": items, "total": total})
```

```
@require_POST

def add_to_cart(request, product_id):
    add_item(request.session, product_id, quantity=1)
    return redirect("cart:cart_detail")
```

```
@require_POST

def remove_from_cart(request, product_id):
    remove_item(request.session, product_id)
    return redirect("cart:cart_detail")
```

```
@require_POST

def update_cart(request, product_id):
    quantity = int(request.POST.get("quantity", 1))
    update_item(request.session, product_id, quantity)
    return redirect("cart:cart_detail")
```

Zen-Zee_style/orders:

orders/admin.py

from django.contrib import admin

from .models import Order, OrderItem

class OrderItemInline(admin.TabularInline):

 model = OrderItem

 extra = 0

@admin.register(Order)

class OrderAdmin(admin.ModelAdmin):

 list_display = ("id", "user", "total_amount", "status", "paid", "created_at")

 list_filter = ("status", "paid")

 search_fields = ("user__username", "id")

 inlines = [OrderItemInline]

orders/apps.py

from django.apps import AppConfig

class OrdersConfig(AppConfig):

 default_auto_field = "django.db.models.BigAutoField"

 name = "orders"

 verbose_name = "Orders"

orders/forms.py

```
from django import forms
```

```
from .models import Order
```

```
class CheckoutForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Order
```

```
        fields = ["address"]
```

```
        widgets = {
```

```
            "address": forms.Textarea(attrs={"rows": 4, "placeholder": "Shipping address"}),
```

```
        }
```

orders/models.py

```
from django.conf import settings
```

```
from django.db import models
```

```
class Order(models.Model):
```

```
    STATUS_PENDING = "pending"
```

```
    STATUS_CONFIRMED = "confirmed"
```

```
    STATUS_SHIPPED = "shipped"
```

```
    STATUS_OUT_FOR_DELIVERY = "out_for_delivery"
```

```
    STATUS_DELIVERED = "delivered"
```

```
    STATUS_CHOICES = [
```

```

        (STATUS_PENDING, "Pending"),
        (STATUS_CONFIRMED, "Confirmed"),
        (STATUS_SHIPPED, "Shipped"),
        (STATUS_OUT_FOR_DELIVERY, "Out for Delivery"),
        (STATUS_DELIVERED, "Delivered"),
    ]

```

```

    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="orders")

```

```

    address = models.TextField()

```

```

    total_amount = models.DecimalField(max_digits=12, decimal_places=2)

```

```

    status = models.CharField(max_length=32, choices=STATUS_CHOICES,
default=STATUS_PENDING)

```

```

    paid = models.BooleanField(default=False)

```

```

    created_at = models.DateTimeField(auto_now_add=True)

```

```

    updated_at = models.DateTimeField(auto_now=True)

```

```

    razorpay_order_id = models.CharField(max_length=128, blank=True, null=True)

```

```

class Meta:

```

```

    ordering = ["-created_at"]

```

```

def __str__(self):

```

```

    return f"Order {self.id} - {self.user.username}"

```

```

class OrderItem(models.Model):

```

```

order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name="items")
product = models.ForeignKey("products.Product", on_delete=models.CASCADE)
seller = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
quantity = models.PositiveIntegerField(default=1)
price = models.DecimalField(max_digits=10, decimal_places=2)

```

```

def __str__(self):
    return f"{self.quantity} x {self.product.name}"

```

```

@property
def subtotal(self):
    return self.price * self.quantity

```

orders/urls.py

```
from django.urls import path
```

```
from . import views
```

```
app_name = "orders"
```

```

urlpatterns = [
    path("checkout/", views.checkout, name="checkout"),
    path("verify/<int:order_id>/", views.verify_payment, name="verify_payment"),
    path("my-orders/", views.order_list, name="order_list"),
    path("my-orders/<int:pk>/", views.order_detail, name="order_detail"),
    path("seller/orders/", views.seller_orders, name="seller_orders"),

```

```

    path("seller/orders/<int:pk>/status/", views.update_order_status,
name="update_order_status"),
]

```

```
orders/views.py
```

```
import os
```

```
from decimal import Decimal
```

```
import razorpay
```

```
from django.contrib import messages
```

```
from django.contrib.auth.decorators import login_required
```

```
from django.shortcuts import get_object_or_404, redirect, render
```

```
from django.urls import reverse
```

```
from orders.models import Order, OrderItem
```

```
from .forms import CheckoutForm
```

```
from cart.utils import get_cart_items, clear_cart
```

```
from products.models import Product
```

```
def _get_razorpay_client():
```

```
    return razorpay.Client(
```

```
        auth=(os.getenv("RAZORPAY_KEY_ID"), os.getenv("RAZORPAY_KEY_SECRET"))
```

```
    )
```

```
@login_required
```

```
def checkout(request):  
    items, total = get_cart_items(request.session)  
  
    if not items:  
        messages.warning(request, "Your cart is empty.")  
        return redirect("products:product_list")  
  
    if request.method == "POST":  
        form = CheckoutForm(request.POST)  
        if form.is_valid():  
            order = form.save(commit=False)  
            order.user = request.user  
            order.total_amount = total  
            order.save()  
            for item in items:  
                OrderItem.objects.create(  
                    order=order,  
                    product=item["product"],  
                    seller=item["product"].seller,  
                    quantity=item["quantity"],  
                    price=item["product"].price,  
                )  
            amount_paise = int(total * Decimal(100))  
            client = _get_razorpay_client()  
            razorpay_order = client.order.create({  
                "amount": amount_paise,  
                "currency": "INR",
```

```

        "receipt": f"order_{order.id}",
    })
    order.razorpay_order_id = razorpay_order.get("id")
    order.save()
    return render(request, "orders/payment.html", {
        "order": order,
        "items": items,
        "total": total,
        "amount_paise": amount_paise,
        "razorpay_key_id": os.getenv("RAZORPAY_KEY_ID"),
        "razorpay_order_id": order.razorpay_order_id,
        "callback_url": request.build_absolute_uri(reverse("orders:verify_payment",
args=[order.id])),
    })
else:
    form = CheckoutForm()

    return render(request, "orders/checkout.html", {"form": form, "items": items, "total": total})

```

@login_required

```

def verify_payment(request, order_id):
    order = get_object_or_404(Order, id=order_id, user=request.user)

    if request.method == "POST":
        payment_id = request.POST.get("razorpay_payment_id")
        razorpay_order_id = request.POST.get("razorpay_order_id")
        signature = request.POST.get("razorpay_signature")

```

```

client = _get_razorpay_client()
try:
    client.utility.verify_payment_signature({
        "razorpay_order_id": razorpay_order_id,
        "razorpay_payment_id": payment_id,
        "razorpay_signature": signature,
    })
    order.paid = True
    order.status = Order.STATUS_CONFIRMED
    order.save()
    clear_cart(request.session)
    messages.success(request, "Payment successful! Your order is confirmed.")
    return redirect("orders:order_detail", pk=order.id)
except razorpay.errors.SignatureVerificationError:
    messages.error(request, "Payment verification failed. Please try again.")
    return redirect("orders:checkout")
messages.error(request, "Invalid payment verification request.")
return redirect("products:product_list")

```

@login_required

```

def order_list(request):
    orders = Order.objects.filter(user=request.user).prefetch_related("items__product")
    return render(request, "orders/order_list.html", {"orders": orders})

```

@login_required

def order_detail(request, pk):

 order = get_object_or_404(Order, pk=pk, user=request.user)

 return render(request, "orders/order_detail.html", {"order": order})

@login_required

def seller_orders(request):

 profile = request.user.profile

 if not profile.is_seller:

 messages.warning(request, "Only sellers can view their orders.")

 return redirect("products:product_list")

 order_items = OrderItem.objects.filter(seller=request.user).select_related("order", "product")

 return render(request, "orders/seller_orders.html", {"order_items": order_items})

@login_required

def update_order_status(request, pk):

 order = get_object_or_404(Order, pk=pk)

 if not request.user.profile.is_seller:

 messages.error(request, "Only sellers can update order status.")

 return redirect("products:product_list")

 if not order.items.filter(seller=request.user).exists():

 messages.error(request, "You do not have permission to update this order.")

```

return redirect("products:product_list")

if request.method == "POST":
    status = request.POST.get("status")
    if status in dict(Order.STATUS_CHOICES):
        order.status = status
        order.save()
        messages.success(request, "Order status updated.")
    return redirect("orders:order_detail", pk=order.id)

```

Zen-Zee_style/products:

products/admin.py

from django.contrib import admin

from .models import Product

@admin.register(Product)

class ProductAdmin(admin.ModelAdmin):

list_display = ("name", "seller", "category", "price", "stock", "created_at")

list_filter = ("category", "seller")

search_fields = ("name", "category", "seller__username")

products/apps.py

from django.apps import AppConfig

```
class ProductsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "products"
    verbose_name = "Products"
```

products/forms.py

```
from django import forms
from .models import Product
```

```
class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "description", "price", "image", "stock", "category"]
        widgets = {
            "description": forms.Textarea(attrs={"rows": 4}),
        }
```

products/models.py

```
from django.conf import settings
from django.db import models
```

```
class Product(models.Model):
    name = models.CharField(max_length=180)
    description = models.TextField(blank=True)
```

```

price = models.DecimalField(max_digits=10, decimal_places=2)

image = models.ImageField(upload_to="products/", blank=True, null=True)

stock = models.PositiveIntegerField(default=0)

category = models.CharField(max_length=120)

seller = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name="products")

created_at = models.DateTimeField(auto_now_add=True)

updated_at = models.DateTimeField(auto_now=True)

```

```

class Meta:

```

```

    ordering = ["-created_at"]

```

```

def __str__(self):

```

```

    return self.name

```

```

products/urls.py

```

```

from django.urls import path

```

```

from . import views

```

```

app_name = "products"

```

```

urlpatterns = [

```

```

    path("", views.product_list, name="product_list"),

```

```

    path("<int:pk>/", views.product_detail, name="product_detail"),

```

```

    path("seller/dashboard/", views.seller_dashboard, name="seller_dashboard"),

```

```

    path("seller/add/", views.product_create, name="product_create"),

```

```

    path("seller/<int:pk>/edit/", views.product_edit, name="product_edit"),
    path("seller/<int:pk>/delete/", views.product_delete, name="product_delete"),
]

```

products/views.py

```
from django.contrib import messages
```

```
from django.contrib.auth.decorators import login_required
```

```
from django.db.models import Q
```

```
from django.shortcuts import get_object_or_404, redirect, render
```

```
from users.models import Profile
```

```
from .forms import ProductForm
```

```
from .models import Product
```

```
def product_list(request):
```

```
    query = request.GET.get("q", "")
```

```
    products = Product.objects.select_related("seller").all()
```

```
    if query:
```

```
        products = products.filter(Q(name__icontains=query) | Q(category__icontains=query))
```

```
    return render(request, "products/product_list.html", {"products": products, "query": query})
```

```
def product_detail(request, pk):
```

```
    product = get_object_or_404(Product, pk=pk)
```

```
    return render(request, "products/product_detail.html", {"product": product})
```

```

@login_required
def seller_dashboard(request):
    profile = request.user.profile
    if not profile.is_seller:
        messages.warning(request, "Only sellers can access the seller dashboard.")
        return redirect("products:product_list")

    products = Product.objects.filter(seller=request.user)
    return render(request, "products/seller_dashboard.html", {"products": products})

```

```

@login_required
def product_create(request):
    profile = request.user.profile
    if not profile.is_seller:
        messages.error(request, "You need seller access to add new products.")
        return redirect("products:product_list")

    if request.method == "POST":
        form = ProductForm(request.POST, request.FILES)
        if form.is_valid():
            product = form.save(commit=False)
            product.seller = request.user
            product.save()
            messages.success(request, "Product added successfully.")

```

```

        return redirect("products:seller_dashboard")
    else:
        form = ProductForm()
    return render(request, "products/product_form.html", {"form": form, "title": "Add Product"})

@login_required
def product_edit(request, pk):
    product = get_object_or_404(Product, pk=pk, seller=request.user)
    if request.method == "POST":
        form = ProductForm(request.POST, request.FILES, instance=product)
        if form.is_valid():
            form.save()
            messages.success(request, "Product updated successfully.")
            return redirect("products:seller_dashboard")
    else:
        form = ProductForm(instance=product)
    return render(request, "products/product_form.html", {"form": form, "title": "Edit Product"})

@login_required
def product_delete(request, pk):
    product = get_object_or_404(Product, pk=pk, seller=request.user)
    if request.method == "POST":
        product.delete()
        messages.success(request, "Product removed.")

```

```

    return redirect("products:seller_dashboard")

return render(request, "products/product_confirm_delete.html", {"product": product})

```

Zen-Zee_style/users:

users/admin.py

```
from django.contrib import admin
```

```
from .models import Profile
```

```
@admin.register(Profile)
```

```
class ProfileAdmin(admin.ModelAdmin):
```

```
    list_display = ("user", "role")
```

```
    list_select_related = ("user",)
```

users/apps.py

```
from django.apps import AppConfig
```

```
class UsersConfig(AppConfig):
```

```
    default_auto_field = "django.db.models.BigAutoField"
```

```
    name = "users"
```

```
    verbose_name = "Users"
```

```
    def ready(self):
```

```
        import users.signals # noqa: F401
```

```
users/forms.py

from allauth.account.forms import SignupForm

from django import forms

from .models import Profile


class SocialSignupForm(SignupForm):

    role = forms.ChoiceField(
        choices=Profile.ROLE_CHOICES,
        widget=forms.RadioSelect,
        initial=Profile.CUSTOMER,
        label="I am a",
    )

    def save(self, request):
        user = super().save(request)

        profile, _ = Profile.objects.get_or_create(user=user)

        profile.role = self.cleaned_data["role"]

        profile.save()

        return user
```

```
class CustomSignupForm(SignupForm):

    role = forms.ChoiceField(
        choices=Profile.ROLE_CHOICES,
        widget=forms.RadioSelect,
```

```

        initial=Profile.CUSTOMER,
        label="I am a",
    )

```

```

def save(self, request):
    user = super().save(request)
    profile, _ = Profile.objects.get_or_create(user=user)
    profile.role = self.cleaned_data["role"]
    profile.save()
    return user

```

```

class ProfileUpdateForm(forms.ModelForm):
    class Meta:
        model = Profile
        fields = ["role"]
        widgets = {
            "role": forms.RadioSelect,
        }

```

users/models.py

```

from django.conf import settings
from django.db import models

```

```

class Profile(models.Model):

```

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

```
CUSTOMER = "customer"
```

```
SELLER = "seller"
```

```
ROLE_CHOICES = [
    (CUSTOMER, "Customer"),
    (SELLER, "Seller"),
]
```

```
user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
```

```
role = models.CharField(max_length=16, choices=ROLE_CHOICES, default=CUSTOMER)
```

```
def __str__(self):
    return f"{self.user.username} ({self.role})"
```

```
@property
```

```
def is_seller(self):
    return self.role == self.SELLER
```

```
users/signals.py
```

```
from django.contrib.auth import get_user_model
```

```
from django.db.models.signals import post_save
```

```
from django.dispatch import receiver
```

```
from .models import Profile
```

```
User = get_user_model()
```

```

@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.get_or_create(user=instance)

users/urls.py

from django.urls import path

from . import views

app_name = "users"

urlpatterns = [
    path("profile/", views.profile, name="profile"),
    path("switch-role/", views.role_switch, name="switch_role"),
]

```

```

users/views.py

from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render
from .forms import ProfileUpdateForm

```

```

@login_required
def profile(request):
    profile = request.user.profile

```

```

if request.method == "POST":
    form = ProfileUpdateForm(request.POST, instance=profile)
    if form.is_valid():
        form.save()
        messages.success(request, "Profile updated successfully.")
        return redirect("users:profile")
    else:
        form = ProfileUpdateForm(instance=profile)

return render(request, "users/profile.html", {"form": form, "profile": profile})

```

@login_required

```

def role_switch(request):
    profile = request.user.profile
    if profile.role == profile.CUSTOMER:
        profile.role = profile.SELLER
        messages.success(request, "Switched to Seller role. You can now access the seller dashboard.")
    else:
        profile.role = profile.CUSTOMER
        messages.success(request, "Switched to Customer role.")
    profile.save()
    return redirect("users:profile")

```

Zen-Zee_style/zenzee:

zenzee/asgi.py

import os

from django.core.asgi import get_asgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "zenzee.settings")

application = get_asgi_application()

zenzee/settings.py

import os

from pathlib import Path

import dj_database_url

from dotenv import load_dotenv

load_dotenv()

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = os.getenv("DJANGO_SECRET_KEY", "replace-this-with-a-secure-key")

DEBUG = os.getenv("DJANGO_DEBUG", "False").lower() in ("true", "1", "yes")

ALLOWED_HOSTS = os.getenv("DJANGO_ALLOWED_HOSTS", "*").split(",")

INSTALLED_APPS = [

"django.contrib.admin",

"django.contrib.auth",

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

```
"django.contrib.contenttypes",
"django.contrib.sessions",
"django.contrib.messages",
"django.contrib.staticfiles",
"django.contrib.sites",

"allauth",
"allauth.account",
"allauth.socialaccount",
"allauth.socialaccount.providers.github",

"users",
"products",
"cart",
"orders",
]
```

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "whitenoise.middleware.WhiteNoiseMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

```

'allauth.account.middleware.AccountMiddleware',
]

ROOT_URLCONF = "zenzee.urls"

TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.debug",
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
                "cart.context_processors.cart_summary",
            ],
        },
    },
]

WSGI_APPLICATION = "zenzee.wsgi.application"

DATABASES = {
    "default": {

```

```

"ENGINE": "django.db.backends.postgresql",
"NAME": os.getenv("POSTGRES_DB", "zenzee"),
"USER": os.getenv("POSTGRES_USER", "zenzee_user"),
"PASSWORD": os.getenv("POSTGRES_PASSWORD", "zenzee_password"),
"HOST": os.getenv("POSTGRES_HOST", "localhost"),
"PORT": os.getenv("POSTGRES_PORT", "5432"),
}
}

if os.environ.get('DATABASE_URL'):
    DATABASES['default'] = dj_database_url.parse(os.environ['DATABASE_URL'])

AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.CommonPasswordValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.NumericPasswordValidator",
    },
]

```

LANGUAGE_CODE = "en-us"

TIME_ZONE = "Asia/Kolkata"

USE_I18N = True

USE_TZ = True

STATIC_URL = "/static/"

STATIC_ROOT = BASE_DIR / "staticfiles"

STATICFILES_DIRS = [BASE_DIR / "static"]

STATICFILES_STORAGE = "whitenoise.storage.CompressedManifestStaticFilesStorage"

MEDIA_URL = "/media/"

MEDIA_ROOT = BASE_DIR / "media"

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"

AUTHENTICATION_BACKENDS = [

 "django.contrib.auth.backends.ModelBackend",

 "allauth.account.auth_backends.AuthenticationBackend",

]

SITE_ID = 1

LOGIN_REDIRECT_URL = "/"

ACCOUNT_LOGOUT_REDIRECT_URL = "/"

ACCOUNT_EMAIL_REQUIRED = True

ACCOUNT_EMAIL_VERIFICATION = "none"

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

```
ACCOUNT_AUTHENTICATION_METHOD = "username"
ACCOUNT_USERNAME_REQUIRED = True
SOCIALACCOUNT_QUERY_EMAIL = True
SOCIALACCOUNT_AUTO_SIGNUP = False
ACCOUNT_FORMS = {"signup": "users.forms.CustomSignupForm"}
SOCIALACCOUNT_FORMS = {"signup": "users.forms.SocialSignupForm"}
```

```
SOCIALACCOUNT_PROVIDERS = {
    "github": {
        "SCOPE": ["read:user", "user:email"],
        "APP": {
            "client_id": os.environ.get("GITHUB_CLIENT_ID"),
            "secret": os.environ.get("GITHUB_CLIENT_SECRET") or
os.environ.get("GITHUB_SECRET"),
            "key": "",
        },
    }
}
```

```
SESSION_COOKIE_SECURE = not DEBUG
CSRF_COOKIE_SECURE = not DEBUG
```

```
RAZORPAY_KEY_ID = os.getenv("RAZORPAY_KEY_ID", "your_key_id")
RAZORPAY_KEY_SECRET = os.getenv("RAZORPAY_KEY_SECRET", "your_key_secret")
```

```
DEFAULT_FROM_EMAIL = os.getenv("DEFAULT_FROM_EMAIL", "noreply@zenzee.style")
```

```
LOGGING = {  
    'version': 1,  
    'disable_existing_loggers': False,  
    'handlers': {  
        'console': {  
            'class': 'logging.StreamHandler',  
        },  
    },  
    'root': {  
        'handlers': ['console'],  
        'level': 'INFO',  
    },  
    'loggers': {  
        'django': {  
            'handlers': ['console'],  
            'level': 'INFO',  
            'propagate': False,  
        },  
    },  
}
```

```

zenzee/urls.py

from django.contrib import admin

from django.urls import include, path

from django.views.generic import RedirectView

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/", include("allauth.urls")),
    path("users/", include(("users.urls", "users"), namespace="users")),
    path("products/", include(("products.urls", "products"), namespace="products")),
    path("cart/", include(("cart.urls", "cart"), namespace="cart")),
    path("orders/", include(("orders.urls", "orders"), namespace="orders")),
    path("", RedirectView.as_view(pattern_name="products:product_list", permanent=False)),
]

```

```

zenzee/wsgi.py

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "zenzee.settings")

application = get_wsgi_application()

```

Zen-Zee_style/templates:

templates/account/login.html

```
{% extends "base.html" %}
```

```
{% load socialaccount %}
```

```
{% block title %}Login{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-md mx-auto rounded-3xl bg-white p-8 shadow-sm">
```

```
<h1 class="text-3xl font-semibold mb-6 text-center">
```

```
    Sign in to Zen-Zee Style
```

```
</h1>
```

```
<form method="post" class="space-y-6">
```

```
    {% csrf_token %}
```

```
    {{ form.as_p }}
```

```
    {% if redirect_field_value %}
```

```
        <input type="hidden" name="redirect_field_value" value="{{ redirect_field_value }}" />
```

```
    {% endif %}
```

```
    <button type="submit" class="w-full rounded-full bg-slate-900 px-6 py-3 text-white font-medium hover:bg-slate-700 transition">Sign in</button>
```

```
</form>
```

```
<div class="my-6 flex items-center gap-3 text-slate-400">
```

```
    <span class="h-px flex-1 bg-slate-200"></span>
```

```

<span>or</span>

<span class="h-px flex-1 bg-slate-200"></span>

</div>

<div class="space-y-4">

  {% providers_media_js %}

  <a href="{% provider_login_url 'github' method='oauth2' %}"

    class="flex items-center justify-center gap-3 rounded-full bg-slate-900 px-6 py-3 text-
white font-medium hover:bg-slate-700 transition">

    <svg xmlns="http://www.w3.org/2000/svg" fill="currentColor" class="w-5 h-5"
viewBox="0 0 24 24">

      <path d="M12 .5C5.65.5 5.65.5 12c0 5.08 3.29 9.39 7.85 10.91.57.178-.25.78-.55 0-
.27-.01-1.17-.02-2.12-3.19.69-3.87-1.54-3.87-1.54-.52-1.32-1.28-1.67-1.28-1.67-1.05-.72.08-
.7.08-.7 1.16.08 1.77 1.2 1.77 1.2 1.03 1.77 2.7 1.26 3.36.96.1-.75.4-1.26.73-1.55-2.55-.29-5.23-
1.27-5.23-5.65 0-1.25.45-2.27 1.2-3.07-.12-.29-.52-1.46.11-3.05 0 0 .98-.31 3.2 1.17a11.1 11.1 0
0 1 5.82 0c2.22-1.48 3.2-1.17 3.2-1.17.63 1.59.23 2.76.11 3.05.75.8 1.2 1.82 1.2 3.07 0 4.39-
2.69 5.36-5.25 5.64.41.35.77 1.04.77 2.1 0 1.52-.01 2.75-.01 3.12 0 .3.2.66.79.55A10.51 10.51 0
0 0 23.5 12C23.5 5.65 18.35.5 12 .5z"/>

    </svg>

    Continue with GitHub

  </a>

</div>

<p class="mt-6 text-center text-sm text-slate-600">

  Don't have an account? <a href="{% url 'account_signup' %}" class="text-slate-900 font-
semibold hover:underline">Sign up</a>

</p>

</div>

{% endblock %}

```

```
templates/account/signup.html
```

```
{% extends "base.html" %}
```

```
{% load socialaccount %}
```

```
{% block title %}Signup{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-md mx-auto rounded-3xl bg-white p-8 shadow-sm">
```

```
<h1 class="text-3xl font-semibold mb-6 text-center">
```

```
    Create your account
```

```
</h1>
```

```
<form method="post" class="space-y-6">
```

```
    {% csrf_token %}
```

```
    {{ form.as_p }}
```

```
    <button type="submit" class="w-full rounded-full bg-slate-900 px-6 py-3 text-white font-
medium hover:bg-slate-700 transition">Sign up</button>
```

```
</form>
```

```
<div class="my-6 flex items-center gap-3 text-slate-400">
```

```
    <span class="h-px flex-1 bg-slate-200"></span>
```

```
    <span>or</span>
```

```
    <span class="h-px flex-1 bg-slate-200"></span>
```

```
</div>
```

```

<div class="space-y-4">

  {% providers_media_js %}

  <a href="{% provider_login_url 'github' method='oauth2' %}"

    class="flex items-center justify-center gap-3 rounded-full bg-slate-900 px-6 py-3 text-
white font-medium hover:bg-slate-700 transition">

    <svg xmlns="http://www.w3.org/2000/svg" fill="currentColor" class="w-5 h-5"
viewBox="0 0 24 24">

      <path d="M12 .5C5.65.5 5.65.5 12c0 5.08 3.29 9.39 7.85 10.91.57.1.78-.25.78-.55 0-
.27-.01-1.17-.02-2.12-3.19.69-3.87-1.54-3.87-1.54-.52-1.32-1.28-1.67-1.28-1.67-1.05-.72.08-
.7.08-.7 1.16.08 1.77 1.2 1.77 1.2 1.03 1.77 2.7 1.26 3.36.96.1-.75.4-1.26.73-1.55-2.55-.29-5.23-
1.27-5.23-5.65 0-1.25.45-2.27 1.2-3.07-.12-.29-.52-1.46.11-3.05 0 0 .98-.31 3.2 1.17a11.1 11.1 0
0 1 5.82 0c2.22-1.48 3.2-1.17 3.2-1.17.63 1.59.23 2.76.11 3.05.75.8 1.2 1.82 1.2 3.07 0 4.39-
2.69 5.36-5.25 5.64.41.35.77 1.04.77 2.1 0 1.52-.01 2.75-.01 3.12 0 .3.2.66.79.55A10.51 10.51 0
0 0 23.5 12C23.5 5.65 18.35.5 12 .5z"/>

    </svg>

    Continue with GitHub

  </a>

</div>

<p class="mt-6 text-center text-sm text-slate-600">

  Already have an account? <a href="{% url 'account_login' %}" class="text-slate-900 font-
semibold hover:underline">Sign in</a>

</p>

</div>

{% endblock %}

```

templates/cart/cart_detail.html

{% extends "base.html" %}

{% block title %}Cart{% endblock %}

{% block content %}

<div class="grid gap-6 lg:grid-cols-4">

<div class="lg:col-span-3 rounded-3xl bg-white p-6 shadow-sm">

<h1 class="text-3xl font-semibold mb-4">Your Cart</h1>

{% if items %}

<div class="space-y-4">

{% for item in items %}

<div class="flex flex-col gap-4 rounded-3xl border border-slate-200 p-4 sm:flex-row sm:items-center sm:justify-between">

<div>

<h2 class="text-xl font-semibold">{{ item.product.name }}</h2>

<p class="text-slate-500">₹{{ item.product.price }} × {{ item.quantity }}</p>

</div>

<div class="flex flex-wrap items-center gap-3">

<form action="{% url 'cart:update_cart' item.product.pk %}" method="post" class="flex items-center gap-2">

{% csrf_token %}

<input type="number" name="quantity" min="1" value="{{ item.quantity }}" class="w-20 rounded-full border border-slate-200 px-3 py-2" />

<button class="rounded-full bg-slate-900 px-4 py-2 text-white hover:bg-slate-700">Update</button>

</form>

```

        <form action="{% url 'cart:remove_from_cart' item.product.pk %}"
method="post">

        {% csrf_token %}

        <button class="rounded-full bg-rose-600 px-4 py-2 text-white hover:bg-rose-
700">Remove</button>

        </form>

    </div>

</div>

{% endfor %}

</div>

{% else %}

    <p class="text-slate-600">Your cart is empty. Browse products to add items.</p>

    {% endif %}

</div>

<aside class="rounded-3xl bg-white p-6 shadow-sm">

    <h2 class="text-xl font-semibold">Summary</h2>

    <p class="mt-3 text-slate-700">Total: <strong>₹{{ total }}</strong></p>

    <a href="{% url 'orders:checkout' %}" class="mt-4 inline-flex w-full items-center justify-
center rounded-full bg-emerald-600 px-4 py-3 text-white hover:bg-emerald-700">Proceed to
Checkout</a>

</aside>

</div>

{% endblock %}

```

templates/includes/navbar.html

```
<header class="bg-white shadow-sm border-b border-slate-200">
```

```
  <div class="max-w-7xl mx-auto px-4 py-4 flex flex-col md:flex-row items-center justify-between gap-4">
```

```
    <div class="flex items-center gap-3">
```

```
      <a href="/" class="text-xl font-semibold text-slate-900">Zen-Zee Style</a>
```

```
      <form action="{% url 'products:product_list' %}" method="get" class="hidden md:flex items-center gap-2">
```

```
        <input name="q" value="{{ request.GET.q }}" class="rounded-full border border-slate-200 px-3 py-2 w-64" placeholder="Search products or categories" />
```

```
        <button class="bg-slate-900 text-white rounded-full px-4 py-2 hover:bg-slate-700">Search</button>
```

```
      </form>
```

```
    </div>
```

```
  <div class="flex items-center gap-4">
```

```
    <a href="{% url 'cart:cart_detail' %}" class="flex items-center gap-2 text-slate-900 hover:text-slate-600">
```

```
      <span>Cart</span>
```

```
      <span class="inline-flex items-center justify-center rounded-full bg-slate-100 px-2 py-1 text-xs font-semibold">₹{{ cart_total_amount|default:'0.00' }}</span>
```

```
    </a>
```

```
    {% if user.is_authenticated %}
```

```
      <a href="{% url 'users:profile' %}" class="text-slate-900 hover:text-slate-600">Profile</a>
```

```
      <a href="{% url 'account_logout' %}" class="text-slate-900 hover:text-slate-600">Logout</a>
```

```
    {% else %}
```

```
      <a href="{% url 'account_login' %}" class="text-slate-900 hover:text-slate-600">Login</a>
```

```

        {% endif %}
    </div>
</div>
</header>

templates/orders/checkout.html

{% extends "base.html" %}

{% block title %}Checkout{% endblock %}

{% block content %}
<div class="grid gap-6 lg:grid-cols-3">
    <div class="lg:col-span-2 rounded-3xl bg-white p-8 shadow-sm">
        <h1 class="text-3xl font-semibold mb-4">Checkout</h1>
        <form method="post" class="space-y-6">
            {% csrf_token %}
            {{ form.as_p }}
            <button class="rounded-full bg-slate-900 px-5 py-3 text-white hover:bg-slate-700">Review Payment</button>
        </form>
    </div>
    <aside class="rounded-3xl bg-white p-6 shadow-sm">
        <h2 class="text-xl font-semibold">Order Summary</h2>
        <div class="mt-4 space-y-3">
            {% for item in items %}
                <div class="flex items-center justify-between text-sm text-slate-600">
                    <span>{{ item.product.name }} × {{ item.quantity }}</span>

```

```

        <span>₹{{ item.subtotal }}</span>

    </div>

    {% endfor %}

</div>

<div class="mt-4 border-t border-slate-200 pt-4 text-lg font-semibold">Total ₹{{ total
}}</div>

</aside>

</div>

{% endblock %}

```

templates/orders/order_detail.html

```
{% extends "base.html" %}
```

```
{% block title %}Order {{ order.id }}{% endblock %}
```

```
{% block content %}
```

```
<div class="grid gap-6 lg:grid-cols-3">
```

```
    <div class="lg:col-span-2 rounded-3xl bg-white p-8 shadow-sm">
```

```
        <h1 class="text-3xl font-semibold mb-4">Order #{{ order.id }}</h1>
```

```
        <p class="text-slate-600 mb-4">Placed on {{ order.created_at|date:"M d, Y H:i" }}</p>
```

```
        <div class="space-y-4">
```

```
            <div class="rounded-3xl bg-slate-50 p-6">
```

```
                <h2 class="text-lg font-semibold mb-2">Delivery Address</h2>
```

```
                <p class="text-slate-700">{{ order.address }}</p>
```

```
            </div>
```

```
            <div class="rounded-3xl bg-slate-50 p-6">
```

```
                <h2 class="text-lg font-semibold mb-3">Order Timeline</h2>
```

```

<div class="space-y-3">

  {% for value,label in order.STATUS_CHOICES %}

    <div class="flex items-center gap-4">

      <div class="h-4 w-4 rounded-full {% if order.status == value %}bg-emerald-600{%
else %}bg-slate-300{% endif %}"></div>

      <span class="{% if order.status == value %}font-semibold text-slate-900{% else
%}text-slate-500{% endif %}">{{ label }}</span>

    </div>

  {% endfor %}

</div>
</div>
<div class="rounded-3xl bg-slate-50 p-6">

  <h2 class="text-lg font-semibold mb-3">Items</h2>

  <div class="space-y-3">

    {% for item in order.items.all %}

      <div class="flex justify-between rounded-2xl bg-white p-4 shadow-sm">

        <div>

          <p class="font-semibold">{{ item.product.name }}</p>

          <p class="text-sm text-slate-500">Qty {{ item.quantity }} • ₹{{ item.price
}}</p>

        </div>

        <p class="font-semibold">₹{{ item.subtotal }}</p>

      </div>

    {% endfor %}

  </div>

</div>
</div>

```

```

</div>

<aside class="rounded-3xl bg-white p-6 shadow-sm">

  <h2 class="text-xl font-semibold mb-4">Summary</h2>

  <p class="mb-2 text-slate-600">Total amount</p>

  <p class="text-3xl font-semibold">₹{{ order.total_amount }}</p>

  <p class="mt-4 text-slate-500">Paid: {{ order.paid | yesno:"Yes,No" }}</p>

  {% if user.profile.is_seller %}

    <form method="post" action="{% url 'orders:update_order_status' order.pk %}"
class="mt-6 space-y-4">

      {% csrf_token %}

      <label class="block text-sm font-medium text-slate-700">Update status</label>

      <select name="status" class="w-full rounded-2xl border border-slate-200 p-3">

        {% for value,label in order.STATUS_CHOICES %}

          <option value="{{ value }}" {% if order.status == value %}selected{% endif %}>{{
label }}</option>

        {% endfor %}

      </select>

      <button class="w-full rounded-full bg-slate-900 px-4 py-3 text-white hover:bg-slate-
700">Save status</button>

    </form>

  {% endif %}

</aside>

</div>

{% endblock %}

```

templates/orders/order_list.html

{% extends "base.html" %}

{% block title %}My Orders{% endblock %}

{% block content %}

<h1 class="text-3xl font-semibold mb-6">My Orders</h1>

<div class="space-y-4">

{% for order in orders %}

<div class="rounded-3xl bg-white p-6 shadow-sm">

<div class="flex flex-col gap-3 sm:flex-row sm:justify-between sm:items-center">

<div>

<h2 class="text-xl font-semibold">Order #{{ order.id }}</h2>

<p class="text-slate-600">Status: {{ order.get_status_display }} • Paid: {{ order.paid|yesno:"Yes,No" }}</p>

</div>

View details

</div>

</div>

{% empty %}

<p class="text-slate-600">No orders yet. Add items to your cart and checkout.</p>

{% endfor %}

</div>

{% endblock %}

templates/orders/payment.html

```
{% extends "base.html" %}
```

```
{% block title %}Pay Now{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-3xl rounded-3xl bg-white p-8 shadow-sm">
```

```
  <h1 class="text-3xl font-semibold mb-4">Complete Payment</h1>
```

```
  <p class="text-slate-600 mb-4">Order #{{ order.id }} total is <strong>₹{{ total }}</strong>. Click below to pay securely with Razorpay.</p>
```

```
  <button id="rzp-button" class="rounded-full bg-emerald-600 px-5 py-3 text-white hover:bg-emerald-700">Pay with Razorpay</button>
```

```
  <form id="payment-form" method="post" action="{{ callback_url }}" class="hidden">
```

```
    {% csrf_token %}
```

```
    <input type="hidden" name="razorpay_payment_id" id="razorpay_payment_id" />
```

```
    <input type="hidden" name="razorpay_order_id" id="razorpay_order_id" value="{{ razorpay_order_id }}" />
```

```
    <input type="hidden" name="razorpay_signature" id="razorpay_signature" />
```

```
  </form>
```

```
</div>
```

```
<script src="https://checkout.razorpay.com/v1/checkout.js"></script>
```

```
<script>
```

```
  const options = {
```

```
    key: "{{ razorpay_key_id }}",
```

```
    amount: {{ amount_paise }},
```

```
    currency: "INR",
```

```
    name: "Zen-Zee Style",
```

```
    description: "Order #{{ order.id }}",
```

```

    order_id: "{{ razorpay_order_id }}",
    handler: function(response) {
        document.getElementById('razorpay_payment_id').value =
response.razorpay_payment_id;

        document.getElementById('razorpay_signature').value = response.razorpay_signature;
        document.getElementById('payment-form').submit();
    },
    theme: { color: "#0f172a" }
};

document.getElementById('rzp-button').addEventListener('click', function(e) {
    e.preventDefault();
    const rzp = new Razorpay(options);
    rzp.open();
});
</script>

```

templates/orders/seller_orders.html

```
{% extends "base.html" %}
```

```
{% block title %}Seller Orders{% endblock %}
```

```
{% block content %}
```

```
<h1 class="text-3xl font-semibold mb-6">Seller Order Activity</h1>
```

```
<div class="space-y-4">
```

```
{% for item in order_items %}
```

```
<div class="rounded-3xl bg-white p-6 shadow-sm">
```

```
<div class="flex flex-col gap-3 sm:flex-row sm:justify-between sm:items-center">
```

```

<div>
    <h2 class="text-xl font-semibold">Order #{{ item.order.id }}</h2>
    <p class="text-slate-600">Product: {{ item.product.name }} • Qty {{ item.quantity
}}</p>
</div>
<div class="text-slate-700">
    <p>Status: {{ item.order.get_status_display }}</p>
    <p class="font-semibold">₹{{ item.subtotal }}</p>
</div>
</div>
</div>
{% empty %}
    <p class="text-slate-600">No orders for your products yet.</p>
{% endfor %}
</div>
{% endblock %}

```

templates/products/product_confirm_delete.html

```
{% extends "base.html" %}
```

```
{% block title %}Delete Product{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-2xl rounded-3xl bg-white p-8 shadow-sm">
```

```
    <h1 class="text-3xl font-semibold mb-4">Delete Product</h1>
```

```
    <p class="mb-6 text-slate-700">Are you sure you want to remove <strong>{{ product.name
}}</strong> from your catalog? This action cannot be undone.</p>

```

```

<form method="post" class="flex gap-4">
  {% csrf_token %}

  <button type="submit" class="rounded-full bg-rose-600 px-5 py-3 text-white hover:bg-rose-700">Confirm Delete</button>

  <a href="{% url 'products:seller_dashboard' %}" class="rounded-full border border-slate-300 px-5 py-3 text-slate-700 hover:bg-slate-50">Cancel</a>

</form>
</div>
{% endblock %}

```

templates/products/product_detail.html

```

{% extends "base.html" %}

{% block title %}{{ product.name }}{% endblock %}

{% block content %}

<div class="grid gap-6 lg:grid-cols-3">

  <div class="lg:col-span-2 rounded-3xl bg-white p-6 shadow-sm">

    <div class="mb-6">

      {% if product.image %}

      {% else %}

        <div class="h-72 rounded-3xl bg-slate-100 flex items-center justify-center text-slate-500">No image</div>

      {% endif %}

    </div>

    <h1 class="text-3xl font-semibold">{{ product.name }}</h1>

```

```

<p class="mt-3 text-slate-700">{{ product.description }}</p>
<div class="mt-4 flex flex-wrap gap-3 text-sm text-slate-500">
    <span>Category: {{ product.category }}</span>
    <span>Seller: {{ product.seller.username }}</span>
    <span>Stock: {{ product.stock }}</span>
</div>
</div>
<div class="rounded-3xl bg-white p-6 shadow-sm">
    <div class="space-y-4">
        <div>
            <span class="text-sm text-slate-500">Price</span>
            <p class="text-3xl font-semibold">₹{{ product.price }}</p>
        </div>
        <form action="{% url 'cart:add_to_cart' product.pk %}" method="post">
            {% csrf_token %}
            <button class="w-full rounded-full bg-emerald-600 px-4 py-3 text-white hover:bg-emerald-700">Add to Cart</button>
        </form>
        <a href="{% url 'products:product_list' %}" class="text-slate-600 hover:text-slate-900">Back to shop</a>
        {% if user.is_authenticated and user.profile.is_seller and user == product.seller %}
            <div class="border-t border-slate-200 pt-4 text-sm text-slate-600">
                <a href="{% url 'products:product_edit' product.pk %}" class="underline">Edit</a> |
                <a href="{% url 'products:product_delete' product.pk %}" class="underline text-rose-600">Delete</a>
            </div>
        {% endif %}
    </div>
</div>

```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
templates/products/product_form.html
```

```
{% extends "base.html" %}
```

```
{% block title %}{{ title }}{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-3xl rounded-3xl bg-white p-8 shadow-sm">
```

```
<h1 class="text-3xl font-semibold mb-6">{{ title }}</h1>
```

```
<form method="post" enctype="multipart/form-data" class="space-y-6">
```

```
{% csrf_token %}
```

```
{{ form.as_p }}
```

```
<button class="rounded-full bg-slate-900 px-5 py-3 text-white hover:bg-slate-700">Save  
Product</button>
```

```
</form>
```

```
</div>
```

```
{% endblock %}
```

```
templates/products/product_list.html
```

```
{% extends "base.html" %}
```

```
{% block title %}Shop{% endblock %}
```

```
{% block content %}
```

```

<div class="flex flex-wrap items-center justify-between gap-4 mb-8">
  <div>
    <h1 class="text-3xl font-semibold">Shop All Products</h1>
    <p class="text-slate-600">Browse curated products from our sellers.</p>
  </div>
  {% if user.is_authenticated and user.profile.is_seller %}
    <a href="{% url 'products:seller_dashboard' %}" class="rounded-full bg-slate-900 px-4 py-2
text-white hover:bg-slate-700">Seller Dashboard</a>
  {% endif %}
</div>

<div class="grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
  {% for product in products %}
    <div class="overflow-hidden rounded-3xl bg-white shadow-sm hover:shadow-md
transition">
      <div class="h-56 bg-slate-100 flex items-center justify-center">
        {% if product.image %}
          
        {% else %}
          <span class="text-slate-500">No image available</span>
        {% endif %}
      </div>
      <div class="p-5">
        <h2 class="text-xl font-semibold">{{ product.name }}</h2>
        <p class="mt-2 text-slate-600">{{ product.category }}</p>
        <p class="mt-4 text-slate-900 font-semibold">₹{{ product.price }}</p>
      </div>
    </div>
  {% endfor %}
</div>

```

```

<div class="mt-5 flex items-center justify-between gap-3">
    <a href="{% url 'products:product_detail' product.pk %}" class="text-slate-900
hover:text-slate-700">View details</a>

    <form action="{% url 'cart:add_to_cart' product.pk %}" method="post">

        {% csrf_token %}

        <button class="rounded-full bg-emerald-600 px-4 py-2 text-white hover:bg-
emerald-700">Add to cart</button>

    </form>

</div>

</div>

</div>

{% empty %}

    <p class="text-slate-600">No products match your search. Try a different keyword.</p>

{% endfor %}

</div>

{% endblock %}

```

templates/products/seller_dashboard.html

```
{% extends "base.html" %}
```

```
{% block title %}Seller Dashboard{% endblock %}
```

```
{% block content %}
```

```
<div class="flex flex-col gap-6">
```

```
    <div class="flex flex-wrap items-center justify-between gap-3 rounded-3xl bg-white p-6
shadow-sm">
```

```
        <div>
```

```
            <h1 class="text-3xl font-semibold">Seller Dashboard</h1>
```

```

    <p class="text-slate-600">Manage your product catalog and track orders for your
items.</p>

</div>

    <a href="{% url 'products:product_create' %}" class="rounded-full bg-slate-900 px-5 py-3
text-white hover:bg-slate-700">Add New Product</a>

</div>

<div class="rounded-3xl bg-white p-6 shadow-sm">

    {% if products %}

        <div class="space-y-4">

            {% for product in products %}

                <div class="flex flex-col gap-3 rounded-3xl border border-slate-200 p-4 md:flex-row
md:items-center md:justify-between">

                    <div>

                        <h2 class="text-xl font-semibold">{{ product.name }}</h2>

                        <p class="text-slate-500">Category: {{ product.category }} • Stock: {{
product.stock }} • ₹{{ product.price }}</p>

                    </div>

                    <div class="flex gap-3">

                        <a href="{% url 'products:product_edit' product.pk %}" class="rounded-full bg-
slate-900 px-4 py-2 text-white hover:bg-slate-700">Edit</a>

                        <a href="{% url 'products:product_delete' product.pk %}" class="rounded-full
bg-rose-600 px-4 py-2 text-white hover:bg-rose-700">Delete</a>

                    </div>

                </div>

            {% endfor %}

        </div>

    {% else %}

```

```
<p class="text-slate-600">You haven't added any products yet. Start by creating a new product.</p>
```

```
{% endif %}
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
templates/socialaccount/signup.html
```

```
{% extends "base.html" %}
```

```
{% block title %}Complete Signup{% endblock %}
```

```
{% block content %}
```

```
<div class="max-w-md mx-auto rounded-3xl bg-white p-8 shadow-sm">
```

```
<h1 class="text-3xl font-semibold mb-4">Complete Your Account</h1>
```

```
<p class="mb-6 text-slate-600">Select your role to finish setting up your Zen-Zee Style account.</p>
```

```
<form method="post" class="space-y-6">
```

```
{% csrf_token %}
```

```
{{ form.as_p }}
```

```
<button class="rounded-full bg-slate-900 px-6 py-3 text-white hover:bg-slate-700">Continue</button>
```

```
</form>
```

```
</div>
```

```
{% endblock %}
```

templates/users/profile.html

```
{% extends "base.html" %}
```

```
{% load static %}
```

```
{% block title %}Profile{% endblock %}
```

```
{% block content %}
```

```
<div class="grid gap-8 md:grid-cols-3">
```

```
  <div class="space-y-4 rounded-3xl bg-white p-6 shadow-sm">
```

```
    <h1 class="text-2xl font-semibold">Profile</h1>
```

```
    <p class="text-sm text-slate-600">Current role: <strong>{{ profile.get_role_display }}</strong></p>
```

```
    <a href="{% url 'users:switch_role' %}" class="inline-flex items-center justify-center rounded-full bg-slate-900 px-4 py-2 text-sm text-white hover:bg-slate-700">Switch Role</a>
```

```
  </div>
```

```
  <div class="md:col-span-2 rounded-3xl bg-white p-6 shadow-sm">
```

```
    <h2 class="text-xl font-semibold mb-4">Update Role</h2>
```

```
    <form method="post" class="space-y-4">
```

```
      {% csrf_token %}
```

```
      {{ form.as_p }}
```

```
      <button class="rounded-full bg-emerald-600 px-5 py-2 text-white hover:bg-emerald-700">Save changes</button>
```

```
    </form>
```

```
  </div>
```

```
</div>
```

```
{% endblock %}
```

templates/base.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8" />
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

```
  <title>Zen-Zee Style | {% block title %}{% endblock %}</title>
```

```
  <script src="https://cdn.tailwindcss.com"></script>
```

```
</head>
```

```
<body class="bg-slate-50 text-slate-900">
```

```
  {% include "includes/navbar.html" %}
```

```
  <main class="max-w-7xl mx-auto px-4 py-8">
```

```
    {% if messages %}
```

```
      <div class="space-y-3 mb-6">
```

```
        {% for message in messages %}
```

```
          <div class="rounded-lg px-4 py-3 text-sm shadow-sm {% if message.tags == 'success' %}bg-emerald-100 text-emerald-900{% elif message.tags == 'error' or message.tags == 'danger' %}bg-rose-100 text-rose-900{% else %}bg-slate-100 text-slate-900{% endif %}">
```

```
            {{ message }}
```

```
          </div>
```

```
        {% endfor %}
```

```
      </div>
```

```
    {% endif %}
```

```
    {% block content %}{% endblock %}
```

```
</main>
```

```
<footer class="bg-slate-900 text-slate-200 py-6 mt-10">
```

```

<div class="max-w-7xl mx-auto px-4 text-center text-sm">
    &copy; 2026 Zen-Zee Style. Built with Django, Tailwind, and Razorpay.
</div>
</footer>
</body>
</html>

```

```
Zen-Zee_style/build.sh
```

```
set -o errexit
```

```
pip install -r requirements.txt
```

```
python manage.py collectstatic --noinput
```

```
python manage.py migrate --noinput
```

```
if [ "$DJANGO_SUPERUSER_PASSWORD" ] && [ "$DJANGO_SUPERUSER_USERNAME" ] && [
"$DJANGO_SUPERUSER_EMAIL" ]; then
```

```
    echo "Creating superuser..."
```

```
    python manage.py shell -c "
```

```
from django.contrib.auth import get_user_model
```

```
User = get_user_model()
```

```
User.objects.filter(username='$DJANGO_SUPERUSER_USERNAME').delete()
```

```
User.objects.create_superuser(
```

```
    username='$DJANGO_SUPERUSER_USERNAME',
```

```
    email='$DJANGO_SUPERUSER_EMAIL',
```

```

        password='$DJANGO_SUPERUSER_PASSWORD'
    )
    print('Superuser created!')
"
fi

```

Zen-Zee_style/ manage.py

```
#!/usr/bin/env python
```

```
import os
```

```
import sys
```

```
if __name__ == "__main__":
```

```
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "zenzee.settings")
```

```
    try:
```

```
        from django.core.management import execute_from_command_line
```

```
    except ImportError as exc:
```

```
        raise ImportError(
```

```
            "Couldn't import Django. Are you sure it's installed and "
```

```
            "available on your PYTHONPATH environment variable? Did you "
```

```
            "forget to activate a virtual environment?"
```

```
        ) from exc
```

```
    execute_from_command_line(sys.argv)
```

Zen-Zee_style/ requirements.txt

Django==5.1.7

gunicorn==21.2.0

uvicorn==0.40.0

whitenoise==6.6.0

psycopg[binary]>=3.2.10

python-decouple==3.8.0

razorpay==1.3.1

setuptools==68.2.2

wheel>=0.44.0

pip>=26.0.0

dj-database-url==2.1.0

django-allauth==65.15.0 # social login providers

PyJWT==2.11.0

python-dotenv>=1.0

cryptography==43.0.3

oauthlib==3.2.2

Pillow==10.4.0

Testing

1. Introduction

Testing is performed to ensure that the Zen-Zee Style e-commerce application functions correctly, meets user requirements, and is free from major defects. The testing process includes functional testing, integration testing, and user interface testing.

2. Testing Objectives

- To verify that all features work as expected
- To identify and fix bugs and errors
- To ensure smooth user experience
- To validate system performance and security

3. Testing Types

3.1 Unit Testing

- Individual components such as models and views are tested independently.

3.2 Integration Testing

- Interaction between modules (e.g., cart → order → payment) is tested.

3.3 System Testing

- Complete system tested as a whole.

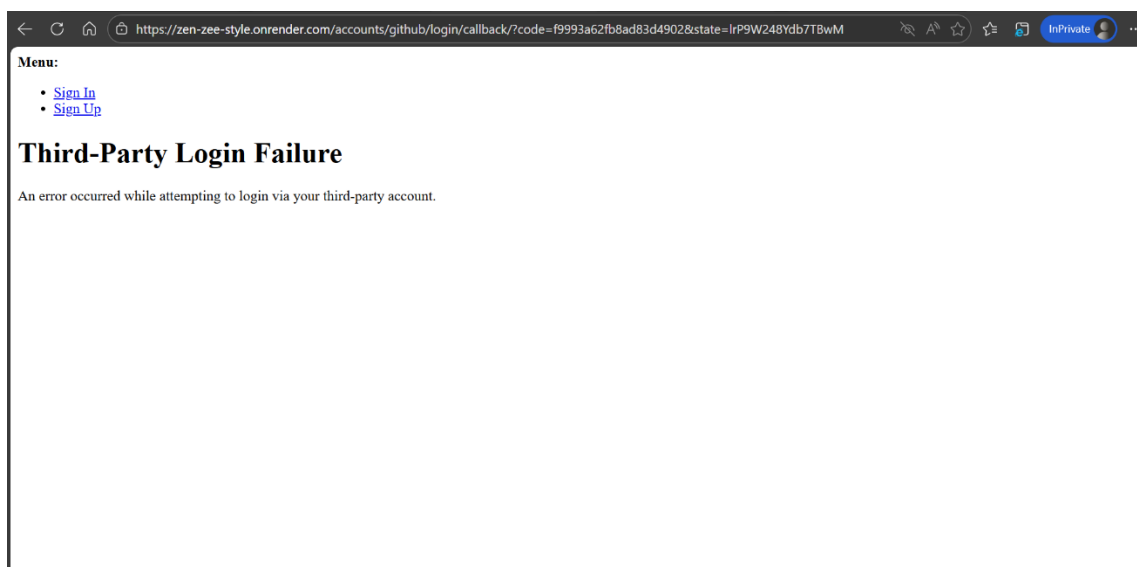
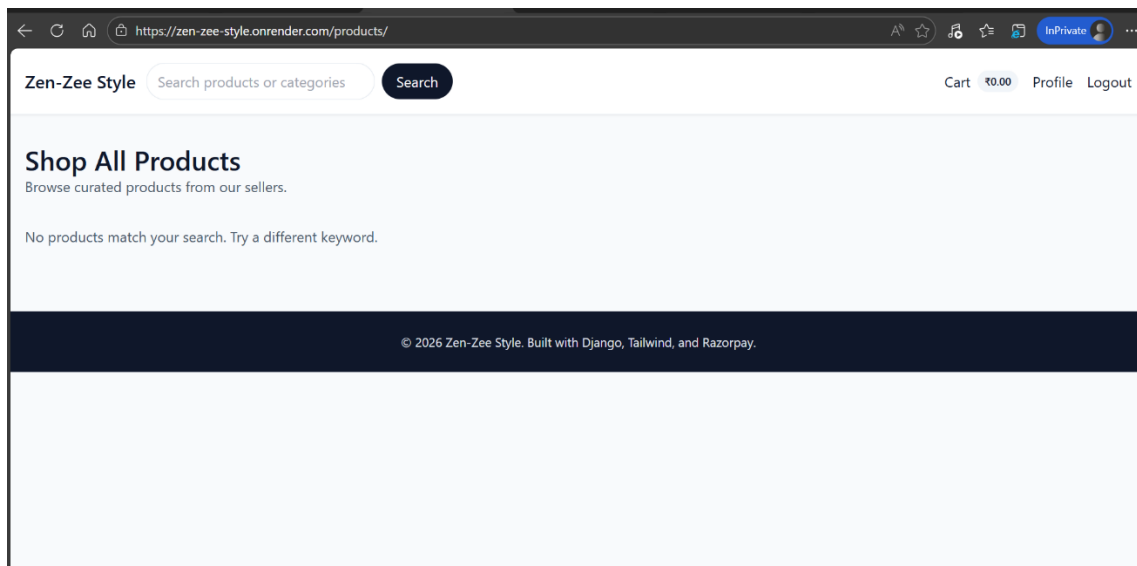
3.4 User Interface Testing

- Ensures proper rendering of UI using Tailwind CSS.

4. Test Cases

4.1 Authentication Testing

Test Case ID	Description	Input	Expected Output	Result
TC01	Login via GitHub	Valid GitHub account	User logged in successfully	Pass
TC02	Unauthorized access	Access without login	Redirect to login page	Pass

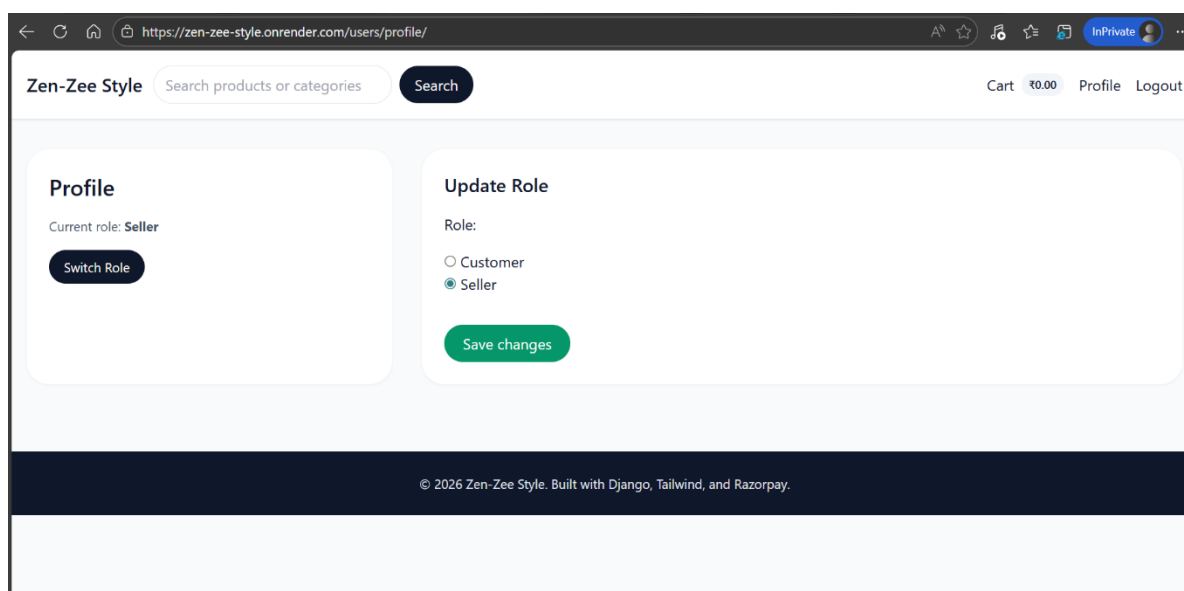
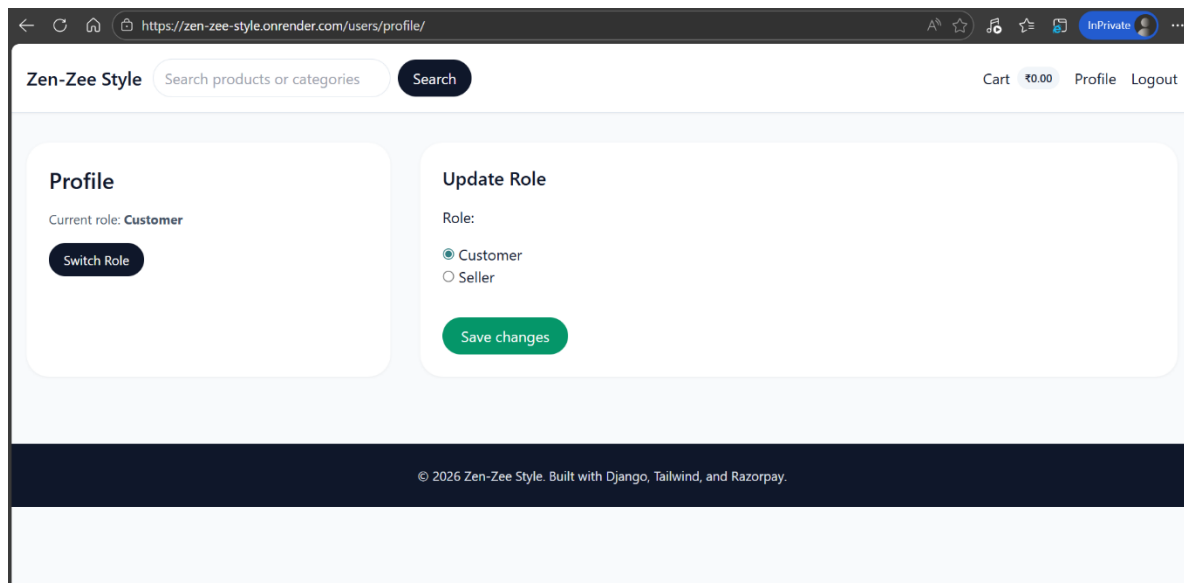


4.2 User Role Testing

Test Case ID	Description	Input	Expected Output	Result
TC03	Select role	Choose Customer/Seller	Role assigned successfully	Pass
TC04	Switch role	Change role in profile	Role updated	Pass
TC05	Seller access control	Customer tries seller dashboard	Dashboard option unavailable	Pass

Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE

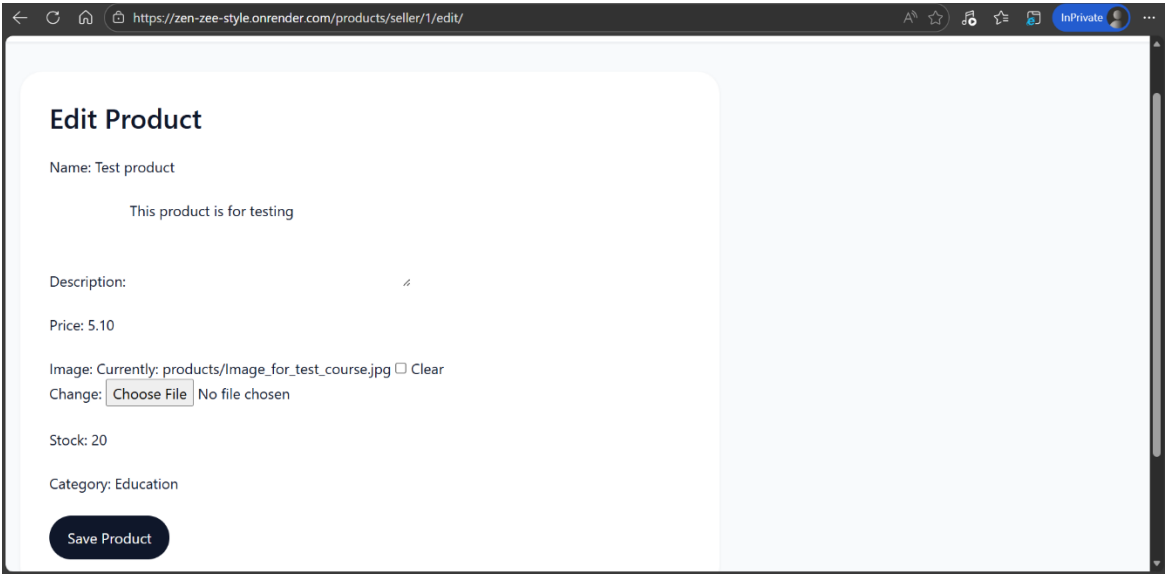
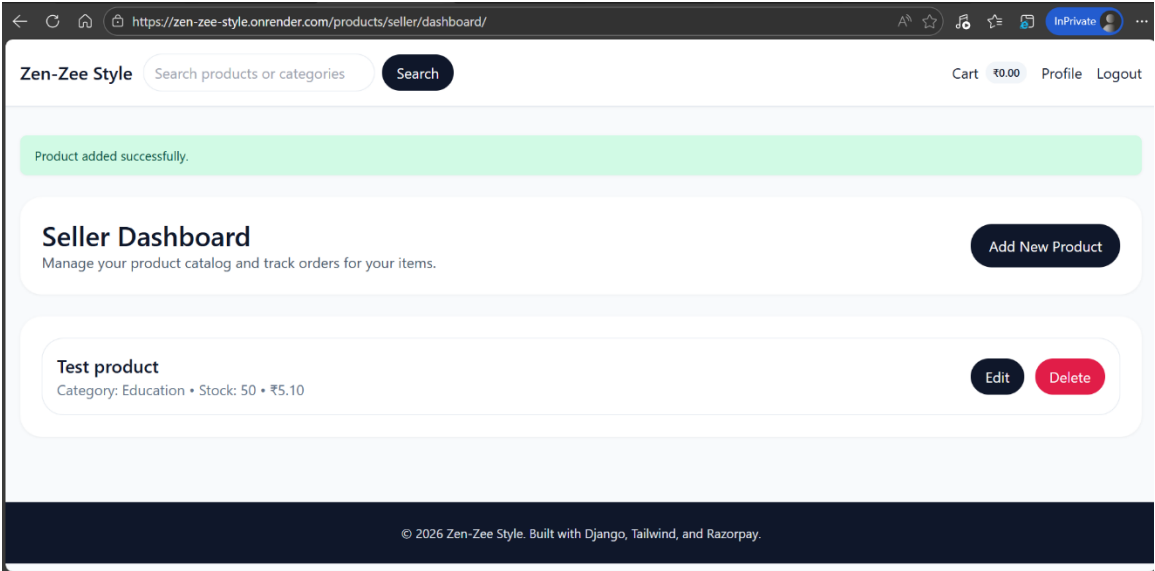


4.3 Product Testing

Test Case ID	Description	Input	Expected Output	Result
TC06	Add product	Valid product data	Product added	Pass
TC07	Edit product	Update product details	Product updated	Pass

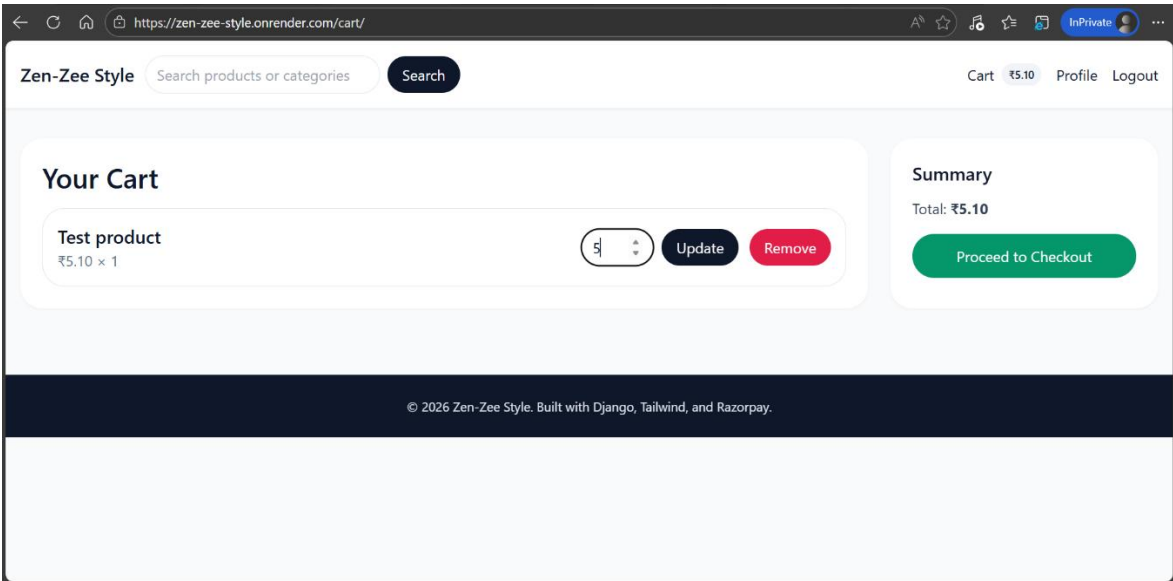
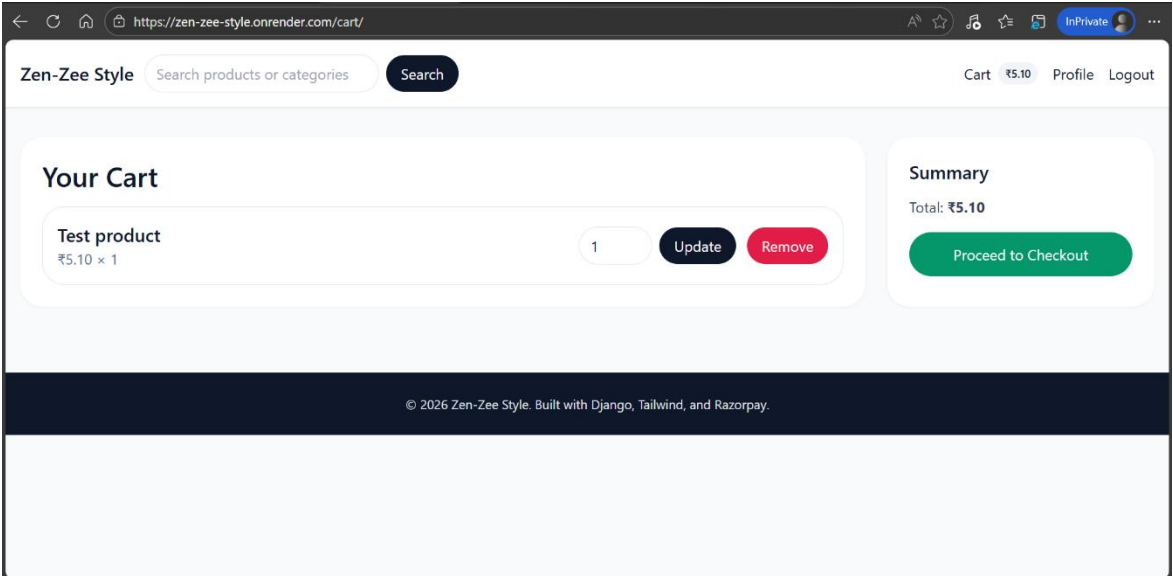
Name: Md Arman Ansari
Enrollment no: 2200279405

Project name: ZEN-ZEE STYLE



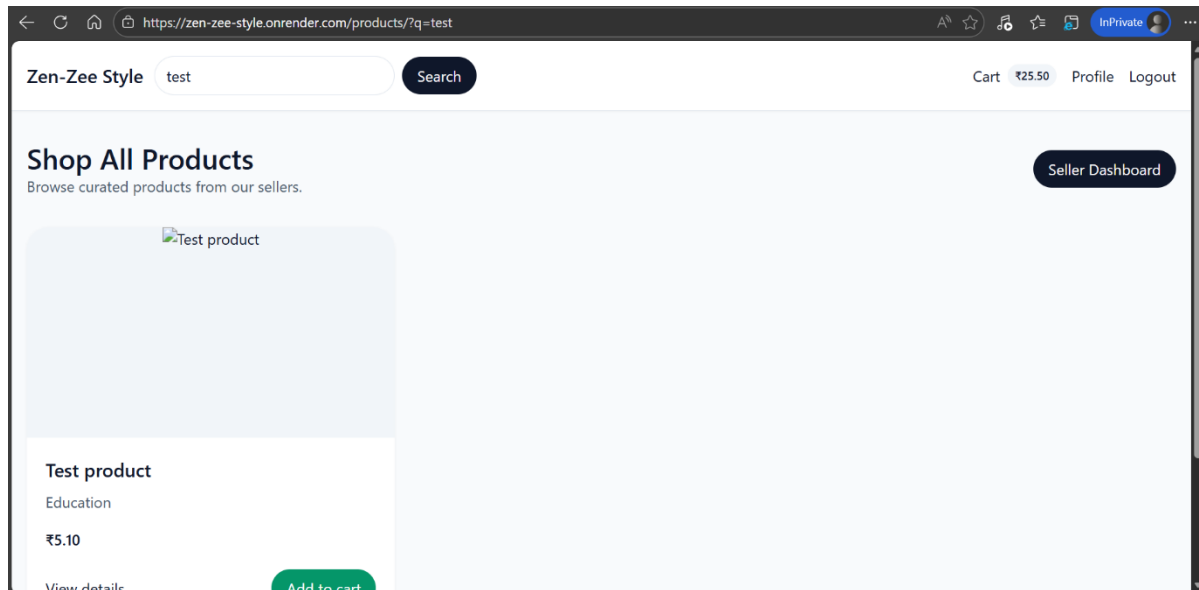
4.4 Cart Testing

Test Case ID	Description	Input	Expected Output	Result
TC09	Add to cart	Select product	Product added to cart	Pass
TC10	Update quantity	Change quantity	Quantity updated	Pass



4.5 Search Testing

Test Case ID	Description	Input	Expected Output	Result
TC18	Search product	Enter keyword	Relevant products displayed	Pass



5. Bug Report (Sample)

Bug ID	Description	Status
B01	GitHub login button not visible	Fixed
B02	Cart not updating properly	Fixed

6. Test Summary

All major functionalities of the Zen-Zee Style application were tested successfully. The system performed as expected with minimal bugs, which were resolved during development.

7. Conclusion

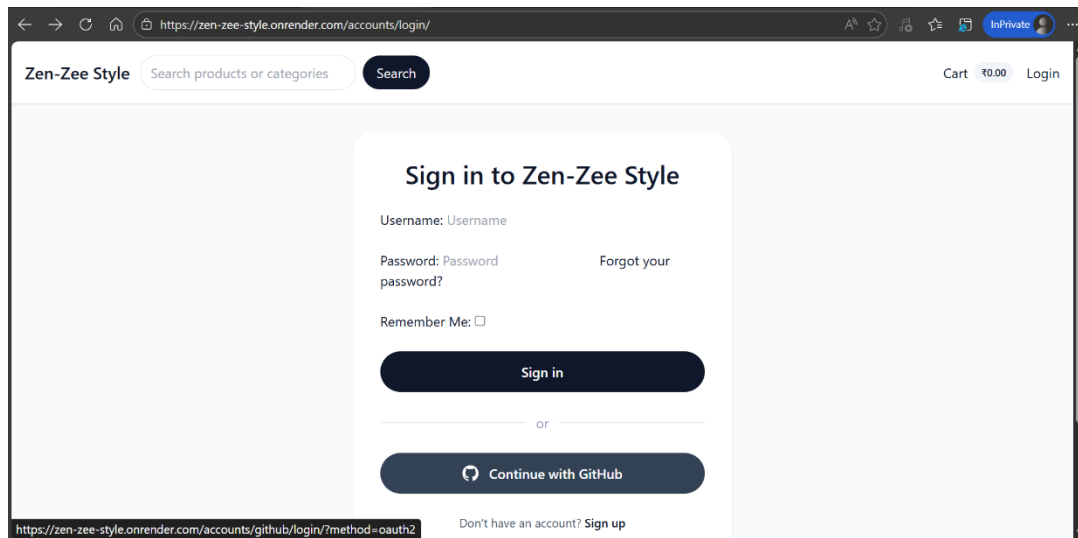
Testing ensured that the application is reliable, functional, and user-friendly. The system meets all specified requirements and is ready for deployment and use.

Input and Output Screens

1. User Authentication

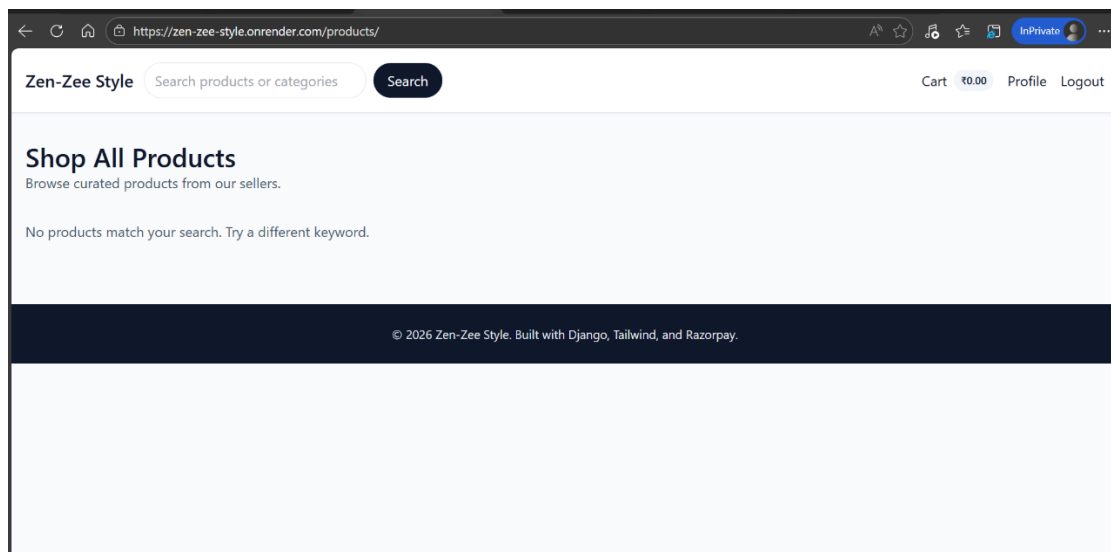
Input Screen:

User clicks on “Continue with GitHub” button to initiate login.



Output Screen:

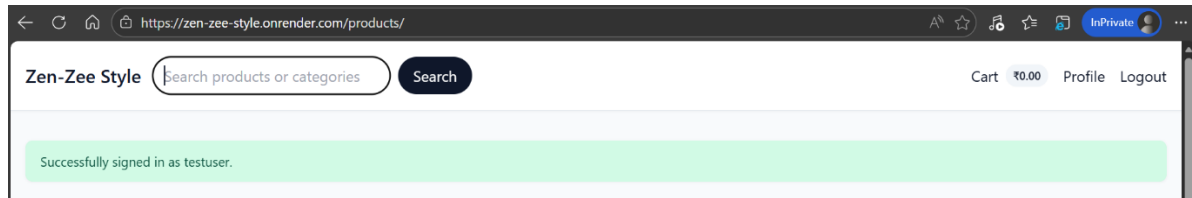
User is successfully logged in and redirected to the homepage.



2. Product Search

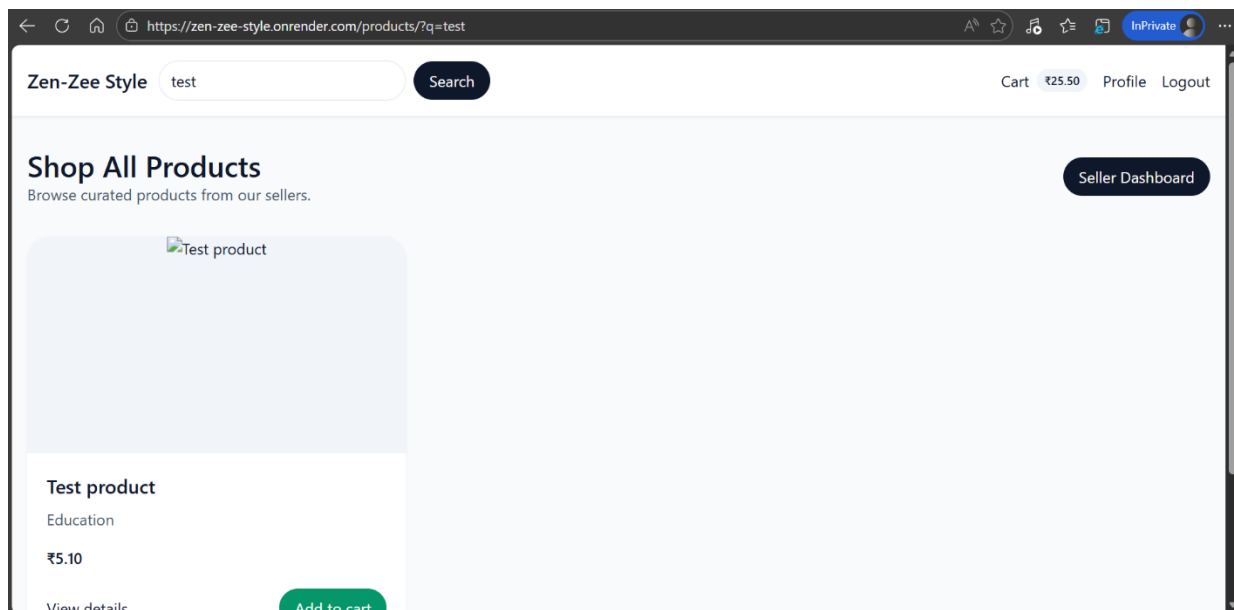
Input Screen:

User enters product name or category in the search bar.



Output Screen:

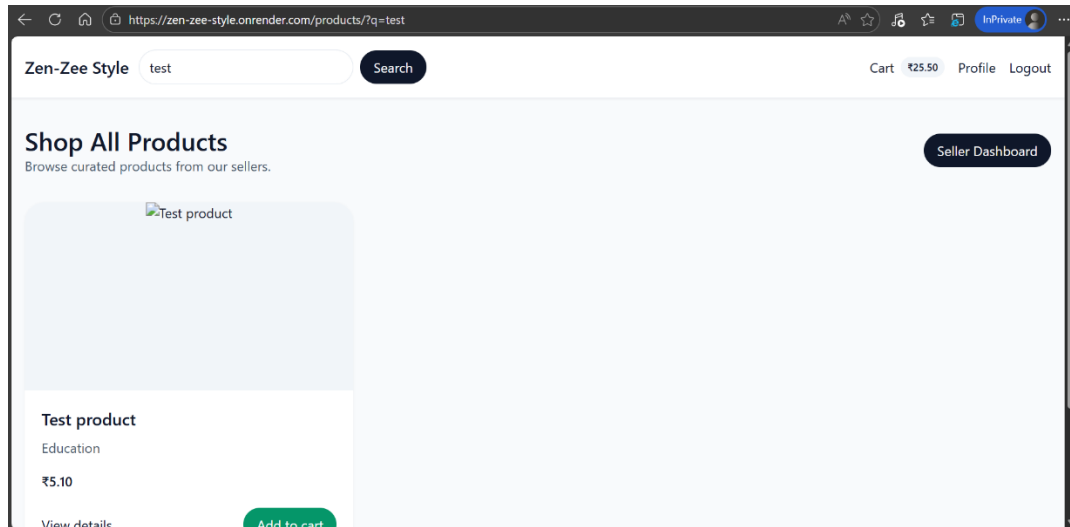
Relevant products are displayed based on search query.



3. Add to Cart

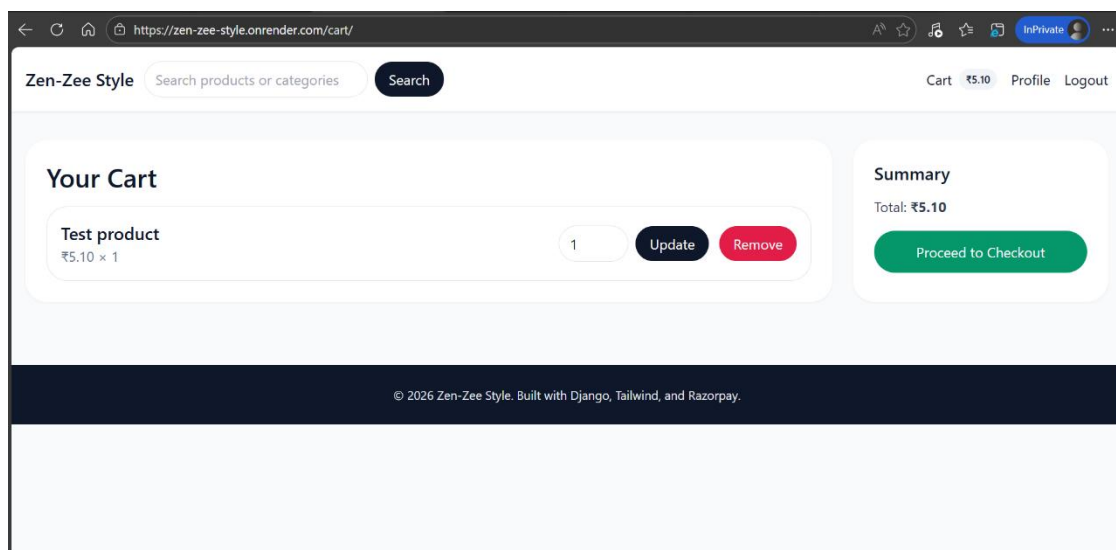
Input Screen:

User selects a product and clicks on “Add to Cart”.



Output Screen:

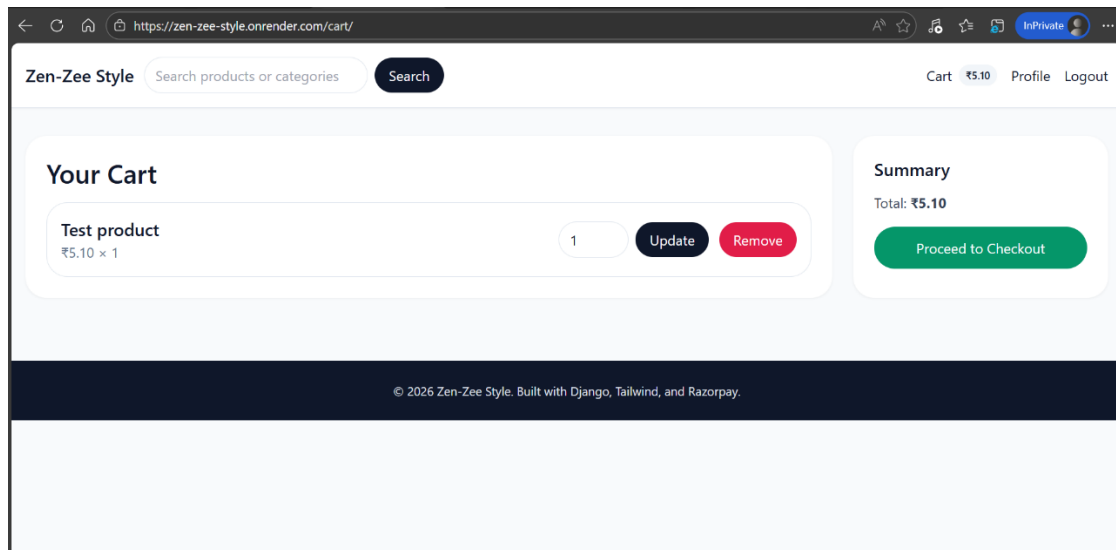
Product is added to cart and reflected in cart page.



4. Cart Management

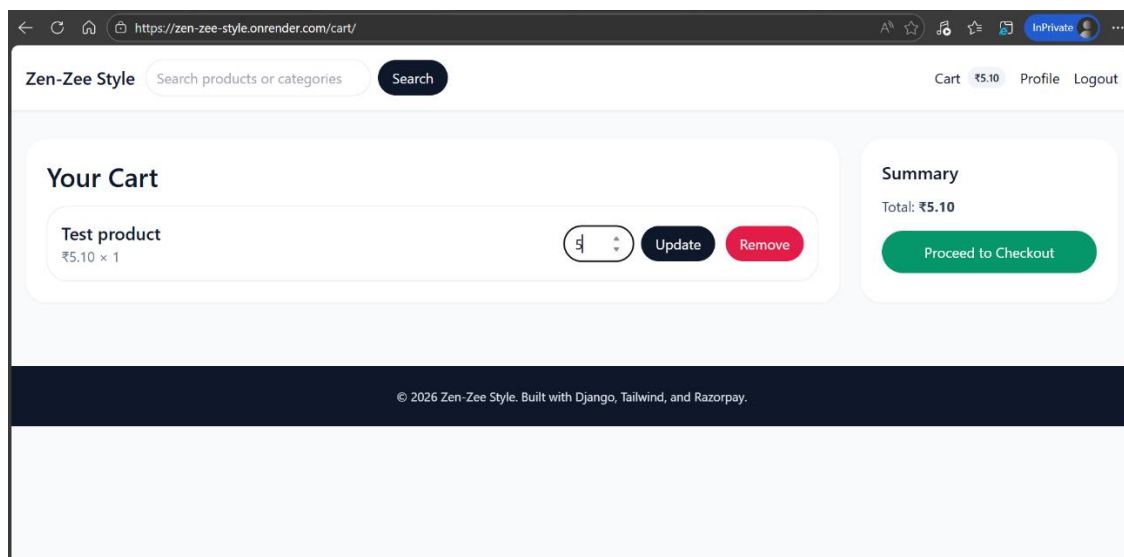
Input Screen:

User updates quantity or removes item from cart.



Output Screen:

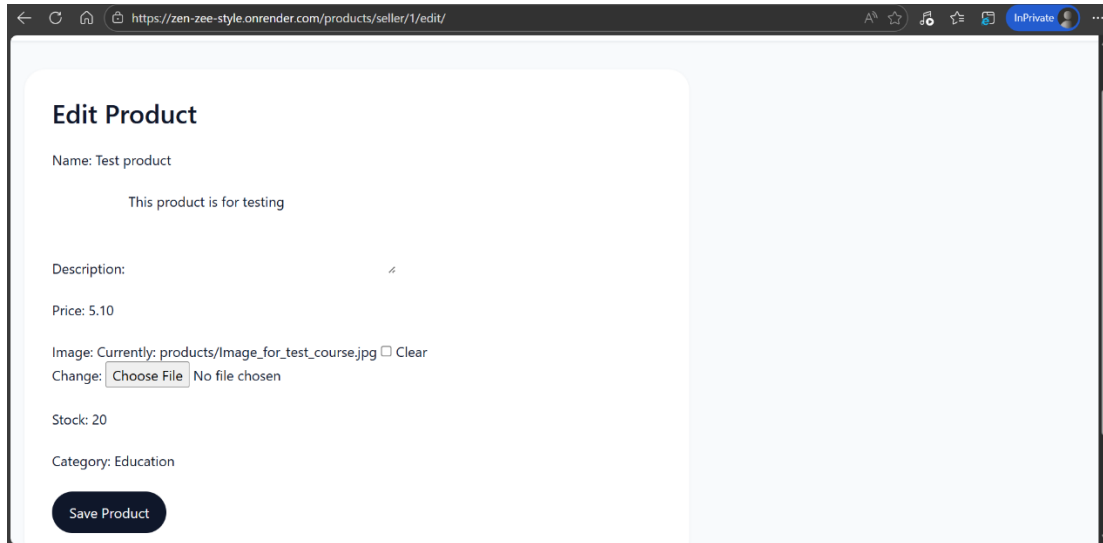
Cart updates dynamically with new total price.



5. Seller Product Management

Input Screen:

Seller adds or edits product details.

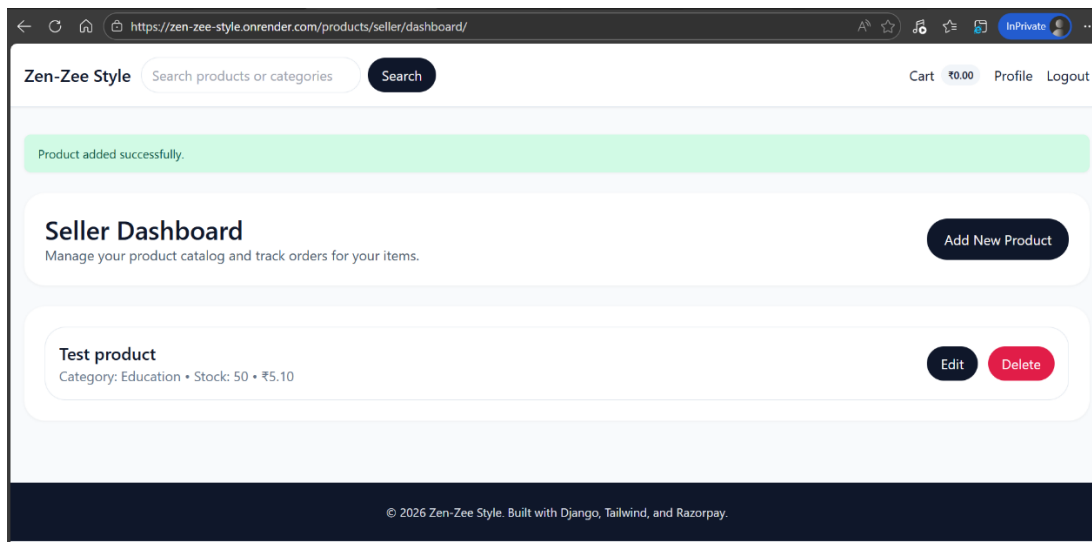


The screenshot shows a web browser window with the URL <https://zen-zee-style.onrender.com/products/seller/1/edit/>. The page is titled "Edit Product". It contains the following fields and controls:

- Name: Test product
- Description: This product is for testing
- Description Editor: A rich text editor with a small icon.
- Price: 5.10
- Image: Currently: products/Image_for_test_course.jpg ☐ Clear
- Change: No file chosen
- Stock: 20
- Category: Education
-

Output Screen:

Product is successfully added/updated in listing.



The screenshot shows the "Seller Dashboard" in the Zen-Zee Style application. The URL is <https://zen-zee-style.onrender.com/products/seller/dashboard/>. The page features a green success message: "Product added successfully." Below this, the "Seller Dashboard" section includes the text "Manage your product catalog and track orders for your items." and an "Add New Product" button. A product listing for "Test product" is shown with details "Category: Education • Stock: 50 • ₹5.10" and "Edit" and "Delete" buttons. The footer contains the text "© 2026 Zen-Zee Style. Built with Django, Tailwind, and Razorpay."

6. Role Selection / Switching

Input Screen:

User selects or switches role (Customer/Seller).

The screenshot shows a web browser window with the URL <https://zen-zee-style.onrender.com/users/profile/>. The page header includes the Zen-Zee Style logo, a search bar, and links for Cart (₹0.00), Profile, and Logout. The main content area is divided into two sections: 'Profile' and 'Update Role'. The 'Profile' section shows the current role as 'Customer' and a 'Switch Role' button. The 'Update Role' section has radio buttons for 'Customer' (selected) and 'Seller', and a 'Save changes' button. The footer contains the copyright notice: © 2026 Zen-Zee Style. Built with Django, Tailwind, and Razorpay.

Output Screen:

System updates role and provides appropriate dashboard access.

The screenshot shows the same web browser window as before, but the role has been updated to 'Seller'. The 'Profile' section now shows 'Current role: Seller' and the 'Switch Role' button. The 'Update Role' section has radio buttons for 'Customer' and 'Seller' (selected), and the 'Save changes' button. The footer remains the same: © 2026 Zen-Zee Style. Built with Django, Tailwind, and Razorpay.

Conclusion

The input and output screens demonstrate the interaction between the user and the system. Each feature has been tested and validated through corresponding input actions and system-generated outputs.

COCOMO Model

1. Introduction

The Constructive Cost Model (COCOMO) is used to estimate the effort, development time, and cost required for software development. The Zen-Zee Style project is categorized under the **Organic Mode** as it is a relatively small and less complex application developed by a small team.

2. Project Classification

The project falls under:

- **Organic Mode**
 - Small team size
 - Less complex requirements
 - Familiar development environment

3. Assumptions

- Estimated project size: **10 KLOC (thousand lines of code)**
- Development mode: Organic

4. COCOMO Formula (Basic Model)

Effort (E):

$$E = 2.4 \times (\text{KLOC})^{1.05}$$

Development Time (D):

$$D = 2.5 \times (E)^{0.38}$$

5. Calculations

Effort Calculation:

$$E = 2.4 \times (10)^{1.05}$$

$$E \approx 2.4 \times 11.22$$

$$E \approx \mathbf{26.93 \text{ Person-Months}}$$

Development Time Calculation:

$$D = 2.5 \times (26.93)^{0.38}$$

$$D \approx 2.5 \times 3.49$$

$$D \approx \mathbf{8.72 \text{ Months}}$$

6. Results

- **Estimated Effort:** ~27 Person-Months
- **Estimated Development Time:** ~9 Months

7. Interpretation

The COCOMO model estimates that the Zen-Zee Style project would require approximately 27 person-months of effort and around 9 months of development time if developed under standard conditions.

8. Conclusion

The COCOMO model provides a theoretical estimation of project effort and time. Although the project was completed in a shorter duration due to prior experience and rapid development techniques, the model highlights the complexity and scope of a full-scale e-commerce system.

Cost Estimation

1. Introduction

Cost estimation is the process of predicting the financial resources required to develop the Zen-Zee Style e-commerce application. It is based on the effort calculated using the COCOMO model.

2. Basis of Estimation

- Estimated Effort: **26.93 Person-Months**
- Cost per Person-Month: **₹20,000**

3. Cost Calculation

Total Cost Formula:

Total Cost = Effort × Cost per Person-Month

Total Cost = 26.93 × 20,000

Total Cost ≈ **₹5,38,600**

4. Cost Breakdown (Approximate)

Component	Estimated Cost
Development Cost	₹4,50,000
Testing & Maintenance	₹50,000
Deployment & Hosting	₹20,000

Component	Estimated Cost
Miscellaneous	₹18,600
Total	₹5,38,600

5. Interpretation

The estimated cost represents the total expense required to develop the project under standard development conditions. The actual development cost may vary depending on team size, tools used, and development time.

6. Conclusion

The cost estimation provides an approximate financial requirement for the Zen-Zee Style project. It demonstrates that developing a full-scale e-commerce system requires significant resources when developed in a professional environment.

Implementation Of Security

Ensuring the security of user data, course content, and administrative controls is essential for the success and trustworthiness of an online course management system like Shiksha Path. The following outlines the key security mechanisms implemented in this project:

1. Authentication and Authorization

Secure Login System using encrypted passwords.

Only registered users (students/admins) can access their respective dashboards.

Login attempts are logged and rate-limited to prevent brute-force attacks.

Authorization:

Role-based access control (RBAC) ensures that users can only access functionalities permitted to their roles (e.g., admin, student).

Admins have elevated privileges such as course creation, content management, and user control.

2. Password Security

- HTTPS encryption
- Input validation and sanitization
- SQL injection prevention
- XSS protection
- Passwords are hashed using bcrypt or SHA-256 before storing in the database.
- Plain text passwords are never stored or transmitted.
- Strong password policy enforced (minimum length, character types).

3. Data Validation and Sanitization

All inputs (registration forms, course uploads, chatbots, etc.) are sanitized to avoid injection attacks SQL Injection prevention using parameterized queries or ORM techniques.

Cross-Site Scripting (XSS) prevention by escaping user-generated content.

4. Access Control

- Role-based permissions (customer/admin)
- Protected API endpoints
- Secure admin panel

5. Database Security

- Encrypted database connections
- Regular backups
- Access restrictions

LIMITATIONS OF THE PROJECT

1. Basic AI Implementation

- Simple recommendation algorithm
- Rule-based chatbot with limited intelligence
- Requires more user data for better accuracy

2. No Mobile App

- Web-based only
- No native mobile applications
- Limited offline functionality

3. Basic Chatbot

- Cannot handle very complex queries
- Requires training with more data
- No emotion detection

4. Limited Payment Options

- Only 1-2 payment gateways
- No cryptocurrency support
- No EMI options

5. No Multi-language

- English only interface
- Limits accessibility for regional users

6. Virtual Trial Room Not Included

- Current version doesn't have image-based try-on
- Planned for future enhancement
- Requires advanced computer vision skills

7. Basic Analytics

- Simple dashboard metrics
- No predictive analytics
- Limited business intelligence features

8. Security Audit Pending

- Basic security implemented
- Professional security audit required
- Penetration testing not performed

9. Limited Third-Party Integration

- Basic payment gateway only
- No logistics API integration
- No CRM connection

10. Testing Coverage

- Manual testing performed
- Automated testing limited
- Performance testing in development environment only

Conclusion:

These limitations reflect the current scope as a student project. The platform provides a strong foundation that can be enhanced with more time, resources, and advanced technical skills.

FUTURE SCOPE

1. Mobile Application

- Native Android and iOS apps
- Push notifications
- Better mobile experience

2. Virtual Trial Room

- **AI-powered image processing to let customers upload photos and virtually try products**
- Computer vision integration
- Product overlay on user images
- Real-time visualization

3. Advanced AI Features

- Voice shopping assistant
- Visual product search
- Predictive analytics
- Dynamic pricing

4. AR Integration

- 3D product visualization
- AR try-on using camera
- Virtual showroom

5. Social Commerce

- Social media integration
- Share products
- Influencer partnerships
- Live shopping events

6. Enhanced Analytics

- Customer behavior prediction
- Sales forecasting
- Advanced reporting

7. Multi-language Support

- Regional languages
- Multiple currencies
- International shipping

BIBLIOGRAPHY

Books & Study Material:

- Software Engineering by Roger S. Pressman – For understanding SDLC, Feasibility Study, and Testing strategies. (
https://openlibrary.org/books/OL9263535M/Software_Engineering?utm_source=chatgpt.com)
- Database System Concepts by Abraham Silberschatz – For RDBMS design and normalization.
(https://openlibrary.org/books/OL22462529M/Database_system_concepts?utm_source=chatgpt.com)
- Web Technologies by Uttam K. Roy – For understanding the integration of frontend and backend technologies.
(https://openlibrary.org/books/OL28754748M/Web_Technologies?utm_source=chatgpt.com)

Research Articles and Educational Platforms:

- Tutorials Point and GeeksforGeeks – For conceptual understanding and sample code references.
- Medium.com – For insights and community-shared best practices in online course systems.
- Coursera, edX, and Udemy – For analysing existing platforms and gathering ideas for features and enhancements.

Technical Support & Discussions:

- Guide – For solving development-related queries and logic implementation.
- GitHub Repositories – For observing open-source LMS project structures and module inspiration.

Acknowledgments:

This project is developed as part of the BCA program at IGNOU. Special thanks to the project guide, faculty members, and peers for their support and feedback throughout the development process.

All external resources, libraries, and APIs used are properly attributed and their licenses are followed.